

SOLIDWORKS API (.NET VBA C#) — Reference

(116/337): IFeatureManager (part 3 of 9)

SOLIDWORKS API Documentation (converted from CHM)

IFeatureManager (part 3 of 9) (Type)

SOLIDWORKS API Help

[Send comments](#) on this topic.

FeatureExtrusion3 Method (IFeatureManager)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureExtrusion3 Method (IFeatureManager)

Creates an extruded feature.

.NET Syntax

Visual Basic (Declaration)

```
Function FeatureExtrusion3( _  
    ByVal Sd As System.Boolean, _  
    ByVal Flip As System.Boolean, _  
    ByVal Dir As System.Boolean, _  
    ByVal T1 As System.Integer, _  
    ByVal T2 As System.Integer, _  
    ByVal D1 As System.Double, _  
    ByVal D2 As System.Double, _  
    ByVal Dchk1 As System.Boolean, _  
    ByVal Dchk2 As System.Boolean, _  
    ByVal Ddir1 As System.Boolean, _  
    ByVal Ddir2 As System.Boolean, _  
    ByVal Dang1 As System.Double, _  
    ByVal Dang2 As System.Double, _  
    ByVal OffsetReverse1 As System.Boolean, _  
    ByVal OffsetReverse2 As System.Boolean, _  
    ByVal TranslateSurface1 As System.Boolean, _  
    ByVal TranslateSurface2 As System.Boolean, _  
    ByVal Merge As System.Boolean, _  
    ByVal UseFeatScope As System.Boolean, _  
    ByVal UseAutoSelect As System.Boolean, _  
    ByVal T0 As System.Integer, _  
    ByVal StartOffset As System.Double, _  
    ByVal FlipStartOffset As System.Boolean _  
) As Feature
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim Sd As System.Boolean  
Dim Flip As System.Boolean  
Dim Dir As System.Boolean  
Dim T1 As System.Integer  
Dim T2 As System.Integer  
Dim D1 As System.Double  
Dim D2 As System.Double  
Dim Dchk1 As System.Boolean  
Dim Dchk2 As System.Boolean  
Dim Ddir1 As System.Boolean  
Dim Ddir2 As System.Boolean  
Dim Dang1 As System.Double  
Dim Dang2 As System.Double  
Dim OffsetReverse1 As System.Boolean  
Dim OffsetReverse2 As System.Boolean  
Dim TranslateSurface1 As System.Boolean  
Dim TranslateSurface2 As System.Boolean  
Dim Merge As System.Boolean  
Dim UseFeatScope As System.Boolean  
Dim UseAutoSelect As System.Boolean  
Dim T0 As System.Integer  
Dim StartOffset As System.Double  
Dim FlipStartOffset As System.Boolean  
Dim value As Feature
```

```
value = instance.FeatureExtrusion3(Sd, Flip, Dir, T1, T2, D1, D2, Dchk1, Dchk2, Ddir1, Ddir2,
Dang1, Dang2, OffsetReverse1, OffsetReverse2, TranslateSurface1, TranslateSurface2, Merge,
UseFeatScope, UseAutoSelect, T0, StartOffset, FlipStartOffset)
```

C#

```
Feature FeatureExtrusion3(
    System.bool Sd,
    System.bool Flip,
    System.bool Dir,
    System.int T1,
    System.int T2,
    System.double D1,
    System.double D2,
    System.bool Dchk1,
    System.bool Dchk2,
    System.bool Ddir1,
    System.bool Ddir2,
    System.double Dang1,
    System.double Dang2,
    System.bool OffsetReverse1,
    System.bool OffsetReverse2,
    System.bool TranslateSurface1,
    System.bool TranslateSurface2,
    System.bool Merge,
    System.bool UseFeatScope,
    System.bool UseAutoSelect,
    System.int T0,
    System.double StartOffset,
    System.bool FlipStartOffset
)
```

C++/CLI

```
Feature^ FeatureExtrusion3(
    & System.bool Sd,
    & System.bool Flip,
    & System.bool Dir,
    & System.int T1,
    & System.int T2,
    & System.double D1,
    & System.double D2,
    & System.bool Dchk1,
    & System.bool Dchk2,
    & System.bool Ddir1,
    & System.bool Ddir2,
    & System.double Dang1,
    & System.double Dang2,
    & System.bool OffsetReverse1,
    & System.bool OffsetReverse2,
    & System.bool TranslateSurface1,
    & System.bool TranslateSurface2,
    & System.bool Merge,
    & System.bool UseFeatScope,
    & System.bool UseAutoSelect,
    & System.int T0,
    & System.double StartOffset,
    & System.bool FlipStartOffset
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Sd

True for single ended, false for double ended

Flip

True to flip the side to cut

Dir

True to flip the direction of extrusion

T1

Termination type for first end of the extrusion as defined in [swEndConditions_e](#)

T2

Termination type for second end of the extrusion as defined in [swEndConditions_e](#)

D1

Depth of extrusion for first end in meters; offset, if T1 is set to [swEndConditions_e.swEndCondOffsetFromSurface](#)
D2
Depth of extrusion for second end in meters; offset, if T2 is set to [swEndConditions_e.swEndCondOffsetFromSurface](#)
Dchk1
True to allow drafting in the first direction, false to not
Dchk2
True to allow drafting in the second direction, false to not
Ddir1
True for first draft angle to be inward, false to be outward; valid only if Dchk1 is true
Ddir2
True for second draft angle to be inward, false to be outward; valid only if Dchk2 is true
Dang1
Draft angle for first end; valid only if Dchk1 is true
Dang2
Draft angle for second end; valid only if Dchk2 is true
OffsetReverse1
True to offset the first end from another face or plane in a direction away from the sketch, false to offset in a direction toward the sketch; valid only if T1 is set to [swEndConditions_e.swEndCondOffsetFromSurface](#)
OffsetReverse2
True to offset the second end from another face or plane in a direction away from the sketch, false to offset in a direction toward the sketch; valid only if T2 is set to [swEndConditions_e.swEndCondOffsetFromSurface](#)
TranslateSurface1
True if the first end of the extrusion is a translation of the reference surface, false if it has a true offset; valid only if T1 is set to [swEndConditions_e.swEndCondOffsetFromSurface](#)
TranslateSurface2
True if the second end of the extrusion is a translation of the reference surface, false if it has a true offset; valid only if T2 is set to [swEndConditions_e.swEndCondOffsetFromSurface](#)
Merge
True to merge the results in a multibody part, false to not
UseFeatScope
True if the feature only affects selected bodies, false if the feature affects all bodies (see **Remarks**)
UseAutoSelect
True to automatically select all bodies and have the feature affect those bodies, false to select the bodies that the feature affects (see **Remarks**)
T0
Start condition as defined in [swStartConditions_e](#)
StartOffset
Distance from the sketch plane to start the extrude; valid only if T0 is set to [swStartConditions_e.swStartOffset](#)
FlipStartOffset
True to flip the direction of the start offset, false to not; valid only if T0 is set to [swStartConditions_e.swStartOffset](#)

Return Value

[IFeature](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureExtrusion3](#).

Example

[Create Extrusion Feature \(VBA\)](#)
[Create Extrusion Feature \(VB.NET\)](#)
[Create Extrusion Feature \(C#\)](#)

Remarks

The difference between this method and the now obsolete IFeatureManager::FeatureExtrusion2 is that this method supports the selection of surface, face, and plane references for start and end conditions.

Before calling this method to extrude a 3D sketch:

Call IModelDocExtension::SelectByID2 to select...	With selection mark...
3D sketch	0
Extrusion direction edge	16
Start condition reference entity	32
End condition reference entity	1

The default direction for cut operations is opposite the sketch normal. The default direction for boss operations is along the sketch normal. Setting the Dir argument to True reverses the default direction. For double-ended extrusions, Direction 2 is always opposite to Direction 1.

The default sketch normal is the same as the face or plane normal where the sketch was placed. To determine this normal vector, see [IFace2::Normal](#) and [IRefPlane::Transform](#), respectively.

When UseAutoSelect is false, the user must select the bodies that the feature affects. Use a selection mark of 8 for each selected body.

When using cut or cavity features that result in multiple bodies, you cannot select to keep all of the resulting bodies or one or more selected bodies.

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[IExtrudeFeatureData2 Interface](#)
[IFeatureManager::FeatureExtrusionThin2 Method \(\)](#)
[IFeatureManager::FeatureCut3 Method \(\)](#)

Availability

FeatureExtrusionThin Method (IFeatureManager)

[See Also](#)[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureExtrusionThin Method (IFeatureManager)Obsolete. Superseded by [IFeatureManager::FeatureExtrusionThin2](#).

.NET Syntax

Visual Basic (Declaration)

```

Function FeatureExtrusionThin( _
    ByVal Sd As System.Boolean, _
    ByVal Flip As System.Boolean, _
    ByVal Dir As System.Boolean, _
    ByVal T1 As System.Integer, _
    ByVal T2 As System.Integer, _
    ByVal D1 As System.Double, _
    ByVal D2 As System.Double, _
    ByVal Dchk1 As System.Boolean, _
    ByVal Dchk2 As System.Boolean, _
    ByVal Ddir1 As System.Boolean, _
    ByVal Ddir2 As System.Boolean, _
    ByVal Dang1 As System.Double, _
    ByVal Dang2 As System.Double, _
    ByVal OffsetReverse1 As System.Boolean, _
    ByVal OffsetReverse2 As System.Boolean, _
    ByVal TranslateSurface1 As System.Boolean, _
    ByVal TranslateSurface2 As System.Boolean, _
    ByVal Merge As System.Boolean, _
    ByVal Thk1 As System.Double, _
    ByVal Thk2 As System.Double, _
    ByVal EndThk As System.Double, _
    ByVal RevThinDir As System.Integer, _
    ByVal CapEnds As System.Integer, _
    ByVal AddBends As System.Boolean, _
    ByVal BendRad As System.Double, _
    ByVal UseFeatScope As System.Boolean, _
    ByVal UseAutoSelect As System.Boolean _
) As Feature

```

Visual Basic (Usage)



```

Dim instance As IFeatureManager
Dim Sd As System.Boolean
Dim Flip As System.Boolean
Dim Dir As System.Boolean
Dim T1 As System.Integer
Dim T2 As System.Integer
Dim D1 As System.Double
Dim D2 As System.Double
Dim Dchk1 As System.Boolean
Dim Dchk2 As System.Boolean
Dim Ddir1 As System.Boolean
Dim Ddir2 As System.Boolean
Dim Dang1 As System.Double
Dim Dang2 As System.Double
Dim OffsetReverse1 As System.Boolean
Dim OffsetReverse2 As System.Boolean
Dim TranslateSurface1 As System.Boolean
Dim TranslateSurface2 As System.Boolean
Dim Merge As System.Boolean
Dim Thk1 As System.Double
Dim Thk2 As System.Double
Dim EndThk As System.Double
Dim RevThinDir As System.Integer
Dim CapEnds As System.Integer

```

```

Dim AddBends As System.Boolean
Dim BendRad As System.Double
Dim UseFeatScope As System.Boolean
Dim UseAutoSelect As System.Boolean
Dim value As Feature

```

```

value = instance.FeatureExtrusionThin(Sd, Flip, Dir, T1, T2, D1, D2, Dchk1, Dchk2, Ddir1, Ddir2,
Dang1, Dang2, OffsetReverse1, OffsetReverse2, TranslateSurface1, TranslateSurface2, Merge, Thk1,
Thk2, EndThk, RevThinDir, CapEnds, AddBends, BendRad, UseFeatScope, UseAutoSelect)

```

C#

```

Feature FeatureExtrusionThin(
    System.bool Sd,
    System.bool Flip,
    System.bool Dir,
    System.int T1,
    System.int T2,
    System.double D1,
    System.double D2,
    System.bool Dchk1,
    System.bool Dchk2,
    System.bool Ddir1,
    System.bool Ddir2,
    System.double Dang1,
    System.double Dang2,
    System.bool OffsetReverse1,
    System.bool OffsetReverse2,
    System.bool TranslateSurface1,
    System.bool TranslateSurface2,
    System.bool Merge,
    System.double Thk1,
    System.double Thk2,
    System.double EndThk,
    System.int RevThinDir,
    System.int CapEnds,
    System.bool AddBends,
    System.double BendRad,
    System.bool UseFeatScope,
    System.bool UseAutoSelect
)

```

C++/CLI

```

Feature^ FeatureExtrusionThin(
& System.bool Sd,
& System.bool Flip,
& System.bool Dir,
& System.int T1,
& System.int T2,
& System.double D1,
& System.double D2,
& System.bool Dchk1,
& System.bool Dchk2,
& System.bool Ddir1,
& System.bool Ddir2,
& System.double Dang1,
& System.double Dang2,
& System.bool OffsetReverse1,
& System.bool OffsetReverse2,
& System.bool TranslateSurface1,
& System.bool TranslateSurface2,
& System.bool Merge,
& System.double Thk1,
& System.double Thk2,
& System.double EndThk,
& System.int RevThinDir,
& System.int CapEnds,
& System.bool AddBends,
& System.double BendRad,

```

```
& System.bool UseFeatScope,  
& System.bool UseAutoSelect  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Sd

Flip

Dir

T1

T2

D1

D2

Dchk1

Dchk2

Ddir1

Ddir2

Dang1

Dang2

OffsetReverse1

OffsetReverse2

TranslateSurface1

TranslateSurface2

Merge

Thk1

Thk2

EndThk

RevThinDir

CapEnds

AddBends

BendRad

UseFeatScope

UseAutoSelect

Visual Basic for Applications (VBA) Syntax

See [FeatureManager.FeatureExtrusionThin](#).

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)

SOLIDWORKS API Help

[Send comments](#) on this topic.

FeatureExtrusionThin2 Method (IFeatureManager)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureExtrusionThin2 Method (IFeatureManager)

Creates an extruded thin feature.

.NET Syntax

Visual Basic (Declaration)

```
Function FeatureExtrusionThin2( _  
    ByVal Sd As System.Boolean, _  
    ByVal Flip As System.Boolean, _  
    ByVal Dir As System.Boolean, _  
    ByVal T1 As System.Integer, _
```

```

ByVal T2 As System.Integer, _
ByVal D1 As System.Double, _
ByVal D2 As System.Double, _
ByVal Dchk1 As System.Boolean, _
ByVal Dchk2 As System.Boolean, _
ByVal Ddir1 As System.Boolean, _
ByVal Ddir2 As System.Boolean, _
ByVal Dang1 As System.Double, _
ByVal Dang2 As System.Double, _
ByVal OffsetReverse1 As System.Boolean, _
ByVal OffsetReverse2 As System.Boolean, _
ByVal TranslateSurface1 As System.Boolean, _
ByVal TranslateSurface2 As System.Boolean, _
ByVal Merge As System.Boolean, _
ByVal Thk1 As System.Double, _
ByVal Thk2 As System.Double, _
ByVal EndThk As System.Double, _
ByVal RevThinDir As System.Integer, _
ByVal CapEnds As System.Integer, _
ByVal AddBends As System.Boolean, _
ByVal BendRad As System.Double, _
ByVal UseFeatScope As System.Boolean, _
ByVal UseAutoSelect As System.Boolean, _
ByVal T0 As System.Integer, _
ByVal StartOffset As System.Double, _
ByVal FlipStartOffset As System.Boolean _
) As Feature

```

Visual Basic (Usage)



```

Dim instance As IFeatureManager
Dim Sd As System.Boolean
Dim Flip As System.Boolean
Dim Dir As System.Boolean
Dim T1 As System.Integer
Dim T2 As System.Integer
Dim D1 As System.Double
Dim D2 As System.Double
Dim Dchk1 As System.Boolean
Dim Dchk2 As System.Boolean
Dim Ddir1 As System.Boolean
Dim Ddir2 As System.Boolean
Dim Dang1 As System.Double
Dim Dang2 As System.Double
Dim OffsetReverse1 As System.Boolean
Dim OffsetReverse2 As System.Boolean
Dim TranslateSurface1 As System.Boolean
Dim TranslateSurface2 As System.Boolean
Dim Merge As System.Boolean
Dim Thk1 As System.Double
Dim Thk2 As System.Double
Dim EndThk As System.Double
Dim RevThinDir As System.Integer
Dim CapEnds As System.Integer
Dim AddBends As System.Boolean
Dim BendRad As System.Double
Dim UseFeatScope As System.Boolean
Dim UseAutoSelect As System.Boolean
Dim T0 As System.Integer
Dim StartOffset As System.Double
Dim FlipStartOffset As System.Boolean
Dim value As Feature

```

```

value = instance.FeatureExtrusionThin2(Sd, Flip, Dir, T1, T2, D1, D2, Dchk1, Dchk2, Ddir1, Ddir2,
Dang1, Dang2, OffsetReverse1, OffsetReverse2, TranslateSurface1, TranslateSurface2, Merge, Thk1,
Thk2, EndThk, RevThinDir, CapEnds, AddBends, BendRad, UseFeatScope, UseAutoSelect, T0,
StartOffset, FlipStartOffset)

```

C#

```

Feature FeatureExtrusionThin2(
    System.bool Sd,
    System.bool Flip,
    System.bool Dir,
    System.int T1,
    System.int T2,
    System.double D1,
    System.double D2,
    System.bool Dchk1,
    System.bool Dchk2,
    System.bool Ddir1,
    System.bool Ddir2,
    System.double Dang1,
    System.double Dang2,
    System.bool OffsetReverse1,
    System.bool OffsetReverse2,
    System.bool TranslateSurface1,
    System.bool TranslateSurface2,
    System.bool Merge,
    System.double Thk1,
    System.double Thk2,
    System.double EndThk,
    System.int RevThinDir,
    System.int CapEnds,
    System.bool AddBends,
    System.double BendRad,
    System.bool UseFeatScope,
    System.bool UseAutoSelect,
    System.int T0,
    System.double StartOffset,
    System.bool FlipStartOffset
)

```

C++/CLI

```

Feature^ FeatureExtrusionThin2(
& System.bool Sd,
& System.bool Flip,
& System.bool Dir,
& System.int T1,
& System.int T2,
& System.double D1,
& System.double D2,
& System.bool Dchk1,
& System.bool Dchk2,
& System.bool Ddir1,
& System.bool Ddir2,
& System.double Dang1,
& System.double Dang2,
& System.bool OffsetReverse1,
& System.bool OffsetReverse2,
& System.bool TranslateSurface1,
& System.bool TranslateSurface2,
& System.bool Merge,
& System.double Thk1,
& System.double Thk2,
& System.double EndThk,
& System.int RevThinDir,
& System.int CapEnds,
& System.bool AddBends,
& System.double BendRad,
& System.bool UseFeatScope,
& System.bool UseAutoSelect,
& System.int T0,
& System.double StartOffset,
& System.bool FlipStartOffset
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Sd

True for single ended, false for double ended

Flip

Not used

Dir

Reverse direction; True if you want Direction1 to be opposite the default direction (see **Remarks**)

T1

Termination type for first end as defined in [swEndConditions_e](#)

T2

Termination type for second end as defined in [swEndConditions_e](#)

D1

Depth of extrusion for first end in meters

D2

Depth of extrusion for second end in meters

Dchk1

True allows draft angle in first direction, false does not allow drafting

Dchk2

True allows draft angle in second direction, false does not allow drafting

Ddir1

True for first draft angle to be inward, false to be outward

Ddir2

True for second draft angle to be inward, false to be outward

Dang1

First draft angle

Dang2

Second draft angle

OffsetReverse1

If you chose to offset the first end condition from another face or plane, then True specifies offset in direction away from the sketch, false specifies offset from the face or plane in direction toward the sketch

OffsetReverse2

If you chose to offset the second end condition from another face or plane, then True specifies offset in direction away from the sketch, false specifies offset from the face or plane in direction toward the sketch

TranslateSurface1

When you choose swEndcondOffsetFromSurface as the termination type for the first end, then True specifies that the end of the extrusion is a translation of the reference surface, false specifies to use a true offset

TranslateSurface2

When you choose swEndcondOffsetFromSurface as the termination type for the second end, then True specifies that the end of the extrusion is a translation of the reference surface, false specifies to use a true offset

Merge

True to merge the resultant base or boss body into an existing body, false to create a distinct solid body

Thk1

Wall thickness 1 (Mid-plane type uses $(Thk1)/2$ for each direction)

Thk2

Wall thickness 2 (Only used when RevThinType =3)

EndThk

End cap thickness (Only used when CapEnds = 1)

RevThinDir

Thin feature type:

- 0 = One direction
- 1 = One direction reverse
- 2 = Mid-plane
- 3 = Two direction

CapEnds

Cap the ends:

- 0 = no cap
- 1 = cap (base features only)

AddBends

True to add auto bends (open profile base features only), false to not

BendRad

Fillet radii if AddBends = True

UseFeatScope

True if the feature only affects selected bodies, false if the feature affects all bodies

UseAutoSelect

True to automatically select all bodies and have the feature affect those bodies, false to select the bodies the feature affects (see **Remarks**)

T0

Start condition as defined in [swStartConditions_e](#)

StartOffset

If T0 set to swStartOffset, then specify offset value

FlipStartOffset

If T0 set to swStartOffset, then True to flip the direction or false to not

Return Value

Pointer to the [IFeature](#) object

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureExtrusionThin2](#).

Example

[Create Extruded Thin Feature \(VBA\)](#)
[Get Annotations Arrays \(C#\)](#)
[Get Annotations Arrays \(VB.NET\)](#)
[Get Annotations Arrays \(VBA\)](#)

Remarks

The default direction for cut operations is opposite the sketch normal. The default direction for boss operations is along the sketch normal. Setting the Dir argument to True reverses the default direction. For double-ended extrusions, Direction 2 is always opposite of Direction 1.

The default sketch normal is the same as the face or plane normal where the sketch was placed. To determine this normal vector, see [IFace2::Normal](#) and [IRefPlane::Transform](#), respectively.

When UseAutoSelect is false, the user must select the bodies that the feature will affect.

When using cut or cavity features that result in multiple bodies, you cannot select to keep all of the resulting bodies or one or more selected bodies.

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[IExtrudeFeatureData2 Interface](#)
[IFeature::FeatureExtrusion2 Method](#)
[IFeature::FeatureCutThin Method](#)

Availability

SolidWorks 2006 SP2, Revision Number 14.2

SOLIDWORKS API Help

[Send comments on this topic.](#)

FeatureFillet Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureFillet Method (IFeatureManager)

Obsolete. Superseded by [IFeatureManager::FeatureFillet2](#).

.NET Syntax

Visual Basic (Declaration)

```
Function FeatureFillet( _  
    ByVal Options As System.Integer, _  
    ByVal R1 As System.Double, _  
    ByVal Ftyp As System.Integer, _  
    ByVal OverflowType As System.Integer, _  
    ByVal Radii As System.Object, _  
    ByVal SetBackDistances As System.Object, _  
    ByVal PointRadiusArray As System.Object _  
) As System.Object
```

Visual Basic (Usage)



```

Dim instance As IFeatureManager
Dim Options As System.Integer
Dim R1 As System.Double
Dim Ftyp As System.Integer
Dim OverflowType As System.Integer
Dim Radii As System.Object
Dim SetBackDistances As System.Object
Dim PointRadiusArray As System.Object
Dim value As System.Object

```

```

value = instance.FeatureFillet(Options, R1, Ftyp, OverflowType, Radii, SetBackDistances,
PointRadiusArray)

```

C#

```

System.object FeatureFillet(
    System.int Options,
    System.double R1,
    System.int Ftyp,
    System.int OverflowType,
    System.object Radii,
    System.object SetBackDistances,
    System.object PointRadiusArray
)

```

C++/CLI

```

System.Object^ FeatureFillet(
    & System.int Options,
    & System.double R1,
    & System.int Ftyp,
    & System.int OverflowType,
    & System.Object^ Radii,
    & System.Object^ SetBackDistances,
    & System.Object^ PointRadiusArray
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Options

Feature fillet options as defined in [swFeatureFilletOptions_e](#)

R1

Radius; valid only if:

- Ftyp is not [swFeatureFilletType_e.swFeatureFilletType_VariableRadius](#)
- Options is set with [swFeatureFilletOptions_e.swFeatureFilletUniformRadius](#)

Ftyp

Type of fillet as defined in [swFeatureFilletType_e](#) (see **Remarks**)

OverflowType

Control of fillet overflowing onto adjacent surfaces as defined in [swFilletOverFlowType_e](#)

Radii

Array containing the radii for the variable radius fillet; valid only if:

- Ftyp is set to [swFeatureFilletType_e.swFeatureFilletType_VariableRadius](#)
- Options is set with [swFeatureFilletOptions_e.swFeatureFilletVarRadiusType](#)
- Options is not set with [swFeatureFilletOptions_e.swFeatureFilletUniformRadius](#)

SetBackDistances

Array containing setback distances for the fillet along the edge

PointRadiusArray

Array containing control point radii at various points along the length of the edge; valid only if Ftyp is set to [swFeatureFilletType_e.swFeatureFilletType_VariableRadius](#)

Return Value

[IFeature](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager.FeatureFillet](#).

Remarks

Before calling this method to create...	Do this...
Simple fillets	1. Call <code>IModelDocExtension::SelectByID2</code> with Mark = 1 to select the edges to fillet. 2. Specify Ftyp with swFeatureFilletType_e.swFeatureFilletType_Simple.
Face blend fillets	1. Call <code>IModelDocExtension::SelectByID2</code> with: • Mark = 2 to select the first set of faces. • Mark = 4 to select the second set of faces. 2. Specify Ftyp with swFeatureFilletType_e.swFeatureFilletType_Face.
Variable radius fillets	1. Call <code>IModelDocExtension::SelectByID2</code> with Mark = 1 to select the edges to fillet. 2. Call <code>IModelDocExtension::SelectByID2</code> with Mark = 256 to select the control point references along the length of the selected edge, one control point reference for each radius in <code>PointRadiusArray</code>. 3. Specify Ftyp with swFeatureFilletType_e.swFeatureFilletType_VariableRadius. 4. Specify multiple radii in <code>Radii</code>. 5. Specify multiple control point radii in <code>PointRadiusArray</code>. 6. Do not set the Options parameter to swFeatureFilletOptions_e.swFilletUniformRadius.
Full-round fillets	1. Call <code>IModelDocExtension::SelectByID2</code> with: • Mark = 2 to select the first set of side faces. • Mark = 512 to select the set of center faces. • Mark = 4 to select the second set of side faces. 2. Specify Ftyp with swFeatureFilletType_e.swFeatureFilletType_FullRound.
Setback fillets	See the SOLIDWORKS Help for details.

See Also

- [IFeatureManager Interface](#)
- [IFeatureManager Members](#)
- [IFeatureManager::FilletXpertChange Method](#)
- [IFeatureManager::FilletXpertRemove Method](#)
- [IVariableFilletFeatureData2 Interface](#)
- [ISimpleFilletFeatureData2 Interface](#)
- [IFeatureManager::IFeatureFillet Method](#)

Availability

SOLIDWORKS 2005 FCS, Revision Number 13.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FeatureFillet2 Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureFillet2 Method (IFeatureManager)

Obsolete. Superseded by [IFeatureManager::FeatureFillet3](#).

.NET Syntax

Visual Basic (Declaration)

```

Function FeatureFillet2( _
    ByVal Options As System.Integer, _
    ByVal R1 As System.Double, _
    ByVal Rho As System.Double, _
    ByVal Ftyp As System.Integer, _
    ByVal OverflowType As System.Integer, _
    ByVal ConicRhoType As System.Integer, _
    ByVal Radii As System.Object, _
    ByVal RhoArr As System.Object, _
    ByVal SetBackDistances As System.Object, _
    ByVal PointRadiusArray As System.Object, _
    ByVal PointRhoArray As System.Object _
) As System.Object

```

Visual Basic (Usage)

Copy Code

```

Dim instance As IFeatureManager
Dim Options As System.Integer
Dim R1 As System.Double
Dim Rho As System.Double
Dim Ftyp As System.Integer
Dim OverflowType As System.Integer
Dim ConicRhoType As System.Integer
Dim Radii As System.Object
Dim RhoArr As System.Object
Dim SetBackDistances As System.Object
Dim PointRadiusArray As System.Object
Dim PointRhoArray As System.Object
Dim value As System.Object

value = instance.FeatureFillet2(Options, R1, Rho, Ftyp, OverflowType, ConicRhoType, Radii, RhoArr,
SetBackDistances, PointRadiusArray, PointRhoArray)

```

C#

```

System.object FeatureFillet2(
    System.int Options,
    System.double R1,
    System.double Rho,
    System.int Ftyp,
    System.int OverflowType,
    System.int ConicRhoType,
    System.object Radii,
    System.object RhoArr,
    System.object SetBackDistances,
    System.object PointRadiusArray,
    System.object PointRhoArray
)

```

C++/CLI

```

System.Object^ FeatureFillet2(
    & System.int Options,
    & System.double R1,
    & System.double Rho,
    & System.int Ftyp,
    & System.int OverflowType,
    & System.int ConicRhoType,
    & System.Object^ Radii,
    & System.Object^ RhoArr,
    & System.Object^ SetBackDistances,
    & System.Object^ PointRadiusArray,
    & System.Object^ PointRhoArray
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Options

Feature fillet options as defined in [swFeatureFilletOptions_e](#)

R1

Uniform radius of the fillet; valid only if:

- Ftyp != [swFeatureFilletType_e.swFeatureFilletType_VariableRadius](#)
- Options include [swFeatureFilletOptions_e.swFeatureFilletUniformRadius](#)

Rho

Value that determines the conic shape of the fillet:

- Conic rho value [0.05, 0.95], if ConicRhoType = [swFeatureFilletProfileType_e.swFeatureFilletConicRho](#) (see **Remarks**)
- Conic radius value, if ConicRhoType = [swFeatureFilletProfileType_e.swFeatureFilletConicRadius](#)
- Circular radius value, if ConicRhoType = [swFeatureFilletProfileType_e.swFeatureFilletCircular](#)

Ftyp

Type of fillet as defined in [swFeatureFilletType_e](#) (see **Remarks**)

OverflowType

Control of fillet overflowing onto adjacent surfaces as defined in [swFilletOverFlowType_e](#)

ConicRhoType

Conic fillet profile type as defined in [swFeatureFilletProfileType_e](#)

Radii

Array containing the radii for the variable radius fillet; valid only if:

- Ftyp = [swFeatureFilletType_e.swFeatureFilletType_VariableRadius](#)
- Options include [swFeatureFilletOptions_e.swFeatureFilletVarRadiusType](#)
- Options do not include [swFeatureFilletOptions_e.swFeatureFilletUniformRadius](#)

RhoArr

Array of Rho values for the specified ConicRhoType for the variable radius fillet; valid only if:

- Ftyp = [swFeatureFilletType_e.swFeatureFilletType_VariableRadius](#)
- Options include [swFeatureFilletOptions_e.swFeatureFilletVarRadiusType](#)
- Options do not include [swFeatureFilletOptions_e.swFeatureFilletUniformRadius](#)

SetBackDistances

Array containing setback distances along the fillet edge

PointRadiusArray

Array containing radii at various control points along the length of the edge; valid only if Ftyp = [swFeatureFilletType_e.swFeatureFilletType_VariableRadius](#)

PointRhoArray

Array of Rho values for the specified ConicRhoType at various control points along the length of the edge; valid only if Ftyp = [swFeatureFilletType_e.swFeatureFilletType_VariableRadius](#)

Return Value

[IFeature](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureFillet2](#).

Remarks

If the conic rho value specified in parameter, Rho, is not in the valid range, then this method replaces it with the closest value in the valid range. For example, 0.01 is replaced by 0.05, and 0.99 is replaced by 0.95.

Before calling this method to create...	You must...
Simple fillets	1. Call <code>IModelDocExtension::SelectByID2</code> with Mark = 1 to select the edges to fillet. 2. Specify Ftyp with swFeatureFilletType_e.swFeatureFilletType_Simple .
Face blend fillets	1. Call <code>IModelDocExtension::SelectByID2</code> with: • Mark = 2 to select the first set of faces. • Mark = 4 to select the second set of faces. 2. Specify Ftyp with swFeatureFilletType_e.swFeatureFilletType_Face .
Variable radius fillets	1. Call <code>IModelDocExtension::SelectByID2</code> with Mark = 1 to select the edges to fillet. 2. Call <code>IModelDocExtension::SelectByID2</code> with Mark = 256 to select the control point references along the length of the selected edge; one control point reference for each radius in <code>PointRadiusArray</code> . 3. Specify Ftyp with swFeatureFilletType_e.swFeatureFilletType_VariableRadius . 4. Specify multiple radii in <code>Radii</code> . 5. Specify multiple control point radii in <code>PointRadiusArray</code> . 6. Do not include swFeatureFilletOptions_e.swFilletUniformRadius in Options.
Full-round fillets	1. Call <code>IModelDocExtension::SelectByID2</code> with: • Mark = 2 to select the first set of side faces. • Mark = 512 to select the set of center faces. • Mark = 4 to select the second set of side faces. 2. Specify Ftyp with swFeatureFilletType_e.swFeatureFilletType_FullRound .
Setback fillets	See the SOLIDWORKS Help for details.
Conic fillets	See the SOLIDWORKS Help for details.

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[IFeatureManager::IFeatureFillet2 Method \(\)](#)
[ISimpleFilletFeatureData2 Interface](#)
[IVariableFilletFeatureData2 Interface](#)
[IFeatureManager::FilletXpertChange Method \(\)](#)
[IFeatureManager::FilletXpertMakeCorner Method \(\)](#)
[IFeatureManager::FilletXpertRemove Method \(\)](#)

Availability

SOLIDWORKS 2014 FCS, Revision Number 22.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FeatureFillet3 Method (IFeatureManager)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureFillet3 Method (IFeatureManager)

Creates a fillet feature for selected edges and control point references.

.NET Syntax

Visual Basic (Declaration)

```
Function FeatureFillet3( _  
    ByVal Options As System.Integer, _  
    ByVal R1 As System.Double, _  
    ByVal R2 As System.Double, _  
    ByVal Rho As System.Double, _  
    ByVal Ftyp As System.Integer, _  
    ByVal OverflowType As System.Integer, _  
    ByVal ConicRhoType As System.Integer, _  
    ByVal Radii As System.Object, _
```

```

    ByVal Dist2Arr As System.Object, _
    ByVal RhoArr As System.Object, _
    ByVal SetBackDistances As System.Object, _
    ByVal PointRadiusArray As System.Object, _
    ByVal PointDist2Array As System.Object, _
    ByVal PointRhoArray As System.Object _
) As System.Object

```

Visual Basic (Usage)



```

Dim instance As IFeatureManager
Dim Options As System.Integer
Dim R1 As System.Double
Dim R2 As System.Double
Dim Rho As System.Double
Dim Ftyp As System.Integer
Dim OverflowType As System.Integer
Dim ConicRhoType As System.Integer
Dim Radii As System.Object
Dim Dist2Arr As System.Object
Dim RhoArr As System.Object
Dim SetBackDistances As System.Object
Dim PointRadiusArray As System.Object
Dim PointDist2Array As System.Object
Dim PointRhoArray As System.Object
Dim value As System.Object

```

```

value = instance.FeatureFillet3(Options, R1, R2, Rho, Ftyp, OverflowType, ConicRhoType, Radii,
Dist2Arr, RhoArr, SetBackDistances, PointRadiusArray, PointDist2Array, PointRhoArray)

```

C#

```

System.object FeatureFillet3(
    System.int Options,
    System.double R1,
    System.double R2,
    System.double Rho,
    System.int Ftyp,
    System.int OverflowType,
    System.int ConicRhoType,
    System.object Radii,
    System.object Dist2Arr,
    System.object RhoArr,
    System.object SetBackDistances,
    System.object PointRadiusArray,
    System.object PointDist2Array,
    System.object PointRhoArray
)

```

C++/CLI

```

System.Object^ FeatureFillet3(
    & System.int Options,
    & System.double R1,
    & System.double R2,
    & System.double Rho,
    & System.int Ftyp,
    & System.int OverflowType,
    & System.int ConicRhoType,
    & System.Object^ Radii,
    & System.Object^ Dist2Arr,
    & System.Object^ RhoArr,
    & System.Object^ SetBackDistances,
    & System.Object^ PointRadiusArray,
    & System.Object^ PointDist2Array,
    & System.Object^ PointRhoArray
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Options

Feature fillet options as defined in [swFeatureFilletOptions_e](#) (see **Remarks**)

R1

Uniform radius of the symmetric fillet; valid only if:

- Ftyp != swFeatureFilletType_e.swFeatureFilletType_VariableRadius
- Options include swFeatureFilletOptions_e.swFeatureFilletUniformRadius

- OR -

Distance 1 radius of the asymmetric fillet; valid only if:

- Options include swFeatureFilletOptions_e.swFeatureFilletAsymmetric

R2

Distance 2 radius of the asymmetric fillet; valid only if:

- Ftyp != swFeatureFilletType_e.swFeatureFilletType_VariableRadius
- Options include swFeatureFilletOptions_e.swFeatureFilletAsymmetric

Rho

Value that determines the conic rho or radius of the fillet:

- Ftyp != swFeatureFilletType_e.swFeatureFilletType_VariableRadius
- Conic rho value [0.05, 0.95], if ConicRhoType = swFeatureFilletProfileType_e.swFeatureFilletConicRho
- Conic radius value, if ConicRhoType = swFeatureFilletProfileType_e.swFeatureFilletConicRadius

(see **Remarks**)

Ftyp

Type of fillet as defined in [swFeatureFilletType_e](#) (see **Remarks**)

OverflowType

Control of fillet overflowing onto adjacent surfaces as defined in [swFilletOverFlowType_e](#) (see **Remarks**)

ConicRhoType

Fillet cross-section profile as defined in [swFeatureFilletProfileType_e](#); valid only if Options does not include swFeatureFilletOptions_e.swFeatureFilletCurvatureContinuous (see **Remarks**)

Radii

Array containing the radii for selected edges for the symmetric variable radius fillet; valid only if:

- Ftyp = swFeatureFilletType_e.swFeatureFilletType_VariableRadius

- OR -

Array containing the Distance 1 radii for selected edges for the asymmetric variable radius fillet; valid only if:

- Ftyp = swFeatureFilletType_e.swFeatureFilletType_VariableRadius
- Options include swFeatureFilletOptions_e.swFeatureFilletAsymmetric

(see **Remarks**)

Dist2Arr

Array containing the Distance 2 radii for selected edges for the asymmetric variable radius fillet; valid only if:

- Ftyp = swFeatureFilletType_e.swFeatureFilletType_VariableRadius
- Options include swFeatureFilletOptions_e.swFeatureFilletAsymmetric

(see **Remarks**)

RhoArr

Array of conic rho or radius values for the specified ConicRhoType for selected edges for the variable radius fillet; valid only if:

- Conic rho value [0.05, 0.95], if ConicRhoType = swFeatureFilletProfileType_e.swFeatureFilletConicRho
- Conic radius value, if ConicRhoType = swFeatureFilletProfileType_e.swFeatureFilletConicRadius
- Ftyp = swFeatureFilletType_e.swFeatureFilletType_VariableRadius

(see **Remarks**)

SetBackDistances

Array assigning setback distances on edges meeting at a selected fillet corner (see **Remarks**)

PointRadiusArray

Array containing control point radii along the lengths of the selected edges for the symmetric variable radius fillet; valid only if:

- Ftyp = swFeatureFilletType_e.swFeatureFilletType_VariableRadius
- Options include swFeatureFilletOptions_e.swFeatureFilletVarRadiusType

- OR -

Array containing Distance 1 control point radii along the lengths of the selected edges for the asymmetric variable radius fillet; valid only if:

- Ftyp = swFeatureFilletType_e.swFeatureFilletType_VariableRadius
- Options include swFeatureFilletOptions_e.swFeatureFilletAsymmetric

(see **Remarks**)

PointDist2Array

Array containing Distance 2 control point radii along the lengths of the selected edges for the asymmetric variable radius fillet; valid only if:

- Ftyp = swFeatureFilletType_e.swFeatureFilletType_VariableRadius
- Options include swFeatureFilletOptions_e.swFeatureFilletAsymmetric

(see **Remarks**)

PointRhoArray

Array of conic rho or radius values for the specified ConicRhoType at various control points along the lengths of the selected edges for the variable radius fillet; valid only if:

- Conic rho value [0.05, 0.95], if ConicRhoType = swFeatureFilletProfileType_e.swFeatureFilletConicRho
- Conic radius value, if ConicRhoType = swFeatureFilletProfileType_e.swFeatureFilletConicRadius
- Ftyp = swFeatureFilletType_e.swFeatureFilletType_VariableRadius

(see **Remarks**)

Return Value

[IFeature](#)

Example

[Create Variable Radius Asymmetric Elliptical Fillet \(VBA\)](#)
[Create Variable Radius Asymmetric Elliptical Fillet \(VB.NET\)](#)
[Create Variable Radius Asymmetric Elliptical Fillet \(C#\)](#)

Remarks

As of SOLIDWORKS 2020, this method is obsolete for creating:

- constant radius fillets
- offset face chamfers
- face fillets
- face-face chamfers
- full round fillets
- partial fillets
- partial chamfers

To create the fillet/chamfers above, use [IFeatureManager::CreateDefinition](#), [IFeatureManager::CreateFeature](#), [IPartialEdgeFilletData](#), and [ISimpleFilletFeatureData2](#).

All of the parameter arrays of this method scale with the number of selected edges to fillet. Take care to populate and order the arrays meaningfully, or this method will fail.

If the conic rho values of Rho, RhoArray, or PointRhoArray are not in the valid range, then this method replaces them with the closest value in the valid range. For example, 0.01 is replaced by 0.05, and 0.99 is replaced by 0.95.

To create...	You must...
Variable radius fillets	1. Call IModelDocExtension::SelectByID2 with Mark = 1 to select the edges to fillet. 2. Call IModelDocExtension::SelectByID2 with Mark = 256 to select the control point references along the lengths of the selected edges; select one control point reference for each radius in PointRadiusArray. 3. Specify Ftyp with swFeatureFilletType_e.swFeatureFilletType_VariableRadius. 4. (Optional) Specify OverflowType. 5. (Optional) Include swFeatureFilletOptions_e.swFeatureFilletVarRadiusType in Options. 6. (Optional) Include swFeatureFilletOptions_e.swFeatureFilletPropagate in Options. 7. (Optional) Include swFeatureFilletOptions_e.swFeatureFilletCurvatureContinuous in Options. 8. (Optional) If Options does not include swFeatureFilletOptions_e.swFeatureFilletCurvatureContinuous, then include swFeatureFilletOptions_e.swFeatureFilletCornerType in Options. 9. (Optional) Include swFeatureFilletOptions_e.swFeatureFilletKeepFeatures in Options. 10. If a symmetric fillet: - If Options does not include swFeatureFilletOptions_e.swFeatureFilletCurvatureContinuous, then specify ConicRhoType with swFeatureProfileType_e.swFeatureFilletCircular, swFeatureFilletConicRadius, or swFeatureFilletConicRho. - Specify Radii. - If ConicRhoType is not swFeatureFilletProfileType_e.swFeatureFilletCircular, then specify RhoArr with conic rho or radius values. - Specify PointRadiusArray with multiple control point radii, one radius for each control point reference selected in step 2. - If ConicRhoType is not swFeatureFilletProfileType_e.swFeatureFilletCircular, then specify PointRhoArray with conic rho or radius values. 11. If an asymmetric fillet: - If Options does not include swFeatureFilletOptions_e.swFeatureFilletCurvatureContinuous, then specify ConicRhoType with swFeatureProfileType_e.swFeatureFilletCircular or swFeatureFilletConicRho. - Include swFeatureFilletOptions_e.swFeatureFilletAsymmetric in Options. - Specify Radii with the Distance 1 radii. - Specify DistZArray with the Distance 2 radii. - If ConicRhoType is not swFeatureFilletProfileType_e.swFeatureFilletCircular, then specify RhoArr with conic rho or radius values. - Specify PointRadiusArray, one Distance 1 radius for each control point reference selected in step 2. - Specify PointDistZArray, one Distance 2 radius for each control point reference selected in step 2. - If ConicRhoType = swFeatureFilletProfileType_e.swFeatureFilletConicRho, specify PointRhoArray with the conic rho values.
Setback fillets	1. Follow the instructions above to create a variable radius fillet. 2. Call IModelDocExtension::SelectByID2 with Mark = 0 to select the vertex of the fillet corner whose setback parameters you want to set. 3. Specify SetBackDistances and call this method.
- OR -	1. Call this method. 2. Use the setback methods on IVariableFilletFeatureData2 to set the setback distances from setback vertices of multiple fillet corners.

For more information, read the **Variable Size Fillets** topic in the SOLIDWORKS Design user-interface help.

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[IFeatureManager::FilletXpertChange Method \(\)](#)
[IFeatureManager::FilletXpertMakeCorner Method \(\)](#)
[IFeatureManager::FilletXpertRemove Method \(\)](#)

Availability

SOLIDWORKS 2015 FCS, Revision Number 23.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FeatureFillPattern Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureFillPattern Method (IFeatureManager)

Obsolete. See [IFeatureManager::CreateFeature](#) and the Remarks in [IFillPatternFeatureData](#).

.NET Syntax

Visual Basic (Declaration)

```
Function FeatureFillPattern(  
    ByVal PatternLayoutType As System.Integer, _
```

```

ByVal LayoutSpacingType As System.Integer, _
ByVal InstanceSpacing As System.Double, _
ByVal StaggerAngle As System.Double, _
ByVal Margins As System.Double, _
ByVal LoopSpacing As System.Double, _
ByVal NoOfInstances As System.Integer, _
ByVal PatternLayoutPolygonSides As System.Integer, _
ByVal FeaturesToPatternType As System.Integer, _
ByVal CreateSeedCutType As System.Integer, _
ByVal Diameter As System.Double, _
ByVal Dimension As System.Double, _
ByVal Rotation As System.Double, _
ByVal Diagonal As System.Double, _
ByVal CreateSeedCutPolygonSides As System.Integer, _
ByVal OuterRadius As System.Double, _
ByVal InnerRadius As System.Double, _
ByVal FlipShapeDirection As System.Boolean, _
ByVal GeometryPattern As System.Boolean _
) As Feature

```

Visual Basic (Usage)



```

Dim instance As IFeatureManager
Dim PatternLayoutType As System.Integer
Dim LayoutSpacingType As System.Integer
Dim InstanceSpacing As System.Double
Dim StaggerAngle As System.Double
Dim Margins As System.Double
Dim LoopSpacing As System.Double
Dim NoOfInstances As System.Integer
Dim PatternLayoutPolygonSides As System.Integer
Dim FeaturesToPatternType As System.Integer
Dim CreateSeedCutType As System.Integer
Dim Diameter As System.Double
Dim Dimension As System.Double
Dim Rotation As System.Double
Dim Diagonal As System.Double
Dim CreateSeedCutPolygonSides As System.Integer
Dim OuterRadius As System.Double
Dim InnerRadius As System.Double
Dim FlipShapeDirection As System.Boolean
Dim GeometryPattern As System.Boolean
Dim value As Feature

```

```

value = instance.FeatureFillPattern(PatternLayoutType, LayoutSpacingType, InstanceSpacing,
StaggerAngle, Margins, LoopSpacing, NoOfInstances, PatternLayoutPolygonSides,
FeaturesToPatternType, CreateSeedCutType, Diameter, Dimension, Rotation, Diagonal,
CreateSeedCutPolygonSides, OuterRadius, InnerRadius, FlipShapeDirection, GeometryPattern)

```

C#

```

Feature FeatureFillPattern(
    System.int PatternLayoutType,
    System.int LayoutSpacingType,
    System.double InstanceSpacing,
    System.double StaggerAngle,
    System.double Margins,
    System.double LoopSpacing,
    System.int NoOfInstances,
    System.int PatternLayoutPolygonSides,
    System.int FeaturesToPatternType,
    System.int CreateSeedCutType,
    System.double Diameter,
    System.double Dimension,
    System.double Rotation,
    System.double Diagonal,
    System.int CreateSeedCutPolygonSides,
    System.double OuterRadius,
    System.double InnerRadius,

```

```

        System.bool FlipShapeDirection,
        System.bool GeometryPattern
    )

```

C++/CLI

```

Feature^ FeatureFillPattern(
& System.int PatternLayoutType,
& System.int LayoutSpacingType,
& System.double InstanceSpacing,
& System.double StaggerAngle,
& System.double Margins,
& System.double LoopSpacing,
& System.int NoOfInstances,
& System.int PatternLayoutPolygonSides,
& System.int FeaturesToPatternType,
& System.int CreateSeedCutType,
& System.double Diameter,
& System.double Dimension,
& System.double Rotation,
& System.double Diagonal,
& System.int CreateSeedCutPolygonSides,
& System.double OuterRadius,
& System.double InnerRadius,
& System.bool FlipShapeDirection,
& System.bool GeometryPattern
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

PatternLayoutType

Pattern layout type as defined in [swPatternLayoutType_e](#)

LayoutSpacingType

Pattern layout spacing as defined in [swPatternLayoutSpacingType_e](#)

InstanceSpacing

Distance between pattern instance centers; valid always for PatternLayoutType = swPatternLayoutType_e.swPatternLayoutPerforation; valid for all other PatternLayoutTypes only if LayoutSpacingType = swPatternLayoutSpacingType_e.swPatternLayoutTargetSpacing

StaggerAngle

Angle by which to stagger rows of pattern instances; valid only if PatternLayoutType = swPatternLayoutType_e.swPatternLayoutPerforation

Margins

Distance between the fill boundary and the outermost instance in the pattern layout; 0 for no margin

LoopSpacing

Distance between instance centers of adjacent pattern loops; valid for all PatternLayoutTypes except swPatternLayoutType_e.swPatternLayoutPerforation

NoOfInstances

Number of pattern instances per loop or side of the pattern layout; valid only if both are true:

- LayoutSpacingType = swPatternLayoutSpacingType_e.swPatternLayoutInstances
- PatternLayoutType is set to one of the following:
 - swPatternLayoutType_e.swPatternLayoutCircular
 - swPatternLayoutType_e.swPatternLayoutPolygon
 - swPatternLayoutType_e.swPatternLayoutSquare

PatternLayoutPolygonSides

Number of sides in the polygonal layout of the pattern; valid only if PatternLayoutType = swPatternLayoutType_e.swPatternLayoutPolygon

FeaturesToPatternType

Type of objects to pattern as defined in [swFeaturesToPatternType_e](#)

CreateSeedCutType

Pattern seed cut types as defined in [swCreateSeedCutType_e](#)

Diameter

Diameter of the circle-cut seed instance; valid only if both are true:

- FeaturesToPatternType = swFeaturesToPatternType_e.swFeaturesToPatternCreateSeedCut
- CreateSeedCutType = swCreateSeedCutType_e.swCreateSeedCutCircle

Dimension

Length of the side of a diamond-cut or square-cut seed instance; valid only if both are true:

- FeaturesToPatternType = swFeaturesToPatternType_e.swFeaturesToPatternCreateSeedCut
- CreateSeedCutType is set to one of the following:
 - swCreateSeedCutType_e.swCreateSeedCutDiamond
 - swCreateSeedCutType_e.swCreateSeedCutSquare

Rotation

Counterclockwise rotation of the seed instance; valid only if both are true:

- FeaturesToPatternType = swFeaturesToPatternType_e.swFeaturesToPatternCreateSeedCut
- CreateSeedCutType is set to one of the following:
 - swCreateSeedCutType_e.swCreateSeedCutDiamond
 - swCreateSeedCutType_e.swCreateSeedCutPolygon
 - swCreateSeedCutType_e.swCreateSeedCutSquare

Diagonal

Length of the diagonal in the diamond-cut seed instance; valid only if both are true:

- FeaturesToPatternType = swFeaturesToPatternType_e.swFeaturesToPatternCreateSeedCut
- CreateSeedCutType = swCreateSeedCutType_e.swCreateSeedCutDiamond

CreateSeedCutPolygonSides

Number of sides in the polygon-cut seed instance; valid only if both are true:

- FeaturesToPatternType = swFeaturesToPatternType_e.swFeaturesToPatternCreateSeedCut
- CreateSeedCutType = swCreateSeedCutType_e.swCreateSeedCutPolygon

OuterRadius

Radius of circle that circumscribes the polygon-cut seed instance; valid only if both are true:

- FeaturesToPatternType = swFeaturesToPatternType_e.swFeaturesToPatternCreateSeedCut
- CreateSeedCutType = swCreateSeedCutType_e.swCreateSeedCutPolygon

InnerRadius

Radius of circle that inscribes the polygon-cut seed instance; valid only if both are true:

- FeaturesToPatternType = swFeaturesToPatternType_e.swFeaturesToPatternCreateSeedCut
- CreateSeedCutType = swCreateSeedCutType_e.swCreateSeedCutPolygon

FlipShapeDirection

True to reverse the direction of the pattern seed cut; valid only if FeaturesToPatternType = swFeaturesToPatternType_e.swFeaturesToPatternCreateSeedCut

GeometryPattern

True to create the pattern using an exact copy of the seed instance, false to not

Return Value

[Feature](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureFillPattern](#).

Remarks

Insert > Pattern/Mirror > Fill Pattern in the SOLIDWORKS menu opens the Fill Pattern PropertyManager. The parameters in this method correspond to the options in the Fill Pattern PropertyManager.

For all pattern layouts except [swPatternLayoutType_e.swPatternLayoutPerforation](#), the fill pattern is created by positioning a pattern seed instance within the fill boundary and copying the pattern in concentric loops about the seed instance.

See the SOLIDWORKS Help for more information about fill pattern features.

To create a fill pattern:

1. Use [IModelDocExtension::SelectById2](#) or another selection method to select the entities required to create a fill pattern feature.

Select...	For the Fill Pattern's...	Using Mark...
Sketch, face, planar curve on a face, or coplanar faces	Fill boundary	1
Direction reference (e.g., edge or axis)	Pattern direction (if not selected, SOLIDWORKS chooses a direction reference appropriate for the specified layout)	2
Vertex or sketch point	Location of the pattern seed instance (if a point is not selected, the pattern seed instance is placed at the center of the fill boundary region)	3
Features	Pattern objects	4
Faces	Pattern objects	5
Bodies	Pattern objects	6

2. Call this method, specifying the following parameters:
 - PatternLayoutType
 - LayoutSpacingType
 - FeaturesToPatternType
 - GeometryPattern
 - LoopSpacing (for all PatternLayoutTypes except swPatternLayoutType_e.swPatternLayoutPerforation)

Depending on the settings above, other parameters might need to be specified.

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)

Availability

SOLIDWORKS 2014 FCS, Revision Number 22.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FeatureFolderLocation Method (IFeatureManager)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureFolderLocation Method (IFeatureManager)

Gets the folder feature for the specified feature.

.NET Syntax

[Visual Basic \(Declaration\)](#)

```
Function FeatureFolderLocation( _  
    ByVal Feature As Feature _  
) As Feature
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim Feature As Feature  
Dim value As Feature  
  
value = instance.FeatureFolderLocation(Feature)
```

C#

```
Feature FeatureFolderLocation(  
    Feature Feature  
)
```

C++/CLI

```
Feature^ FeatureFolderLocation(  
& Feature^ Feature  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

[Feature](#)
[IFeature](#) whose folder to get

Return Value

[IFeature](#) folder

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureFolderLocation](#).

Example

[Move Assembly Components to New Folder \(VBA\)](#)
[Move Assembly Components to New Folder \(VB.NET\)](#)
[Move Assembly Components to New Folder \(C#\)](#)

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[IFeatureManager::CutListFolderLocation Method \(\)](#)

Availability

SOLIDWORKS 2015 FCS, Revision Number 23.0

SOLIDWORKS API Help

[Send comments](#) on this topic.

FeatureLinearPattern Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureLinearPattern Method (IFeatureManager)

Obsolete. Superseded by [IFeatureManager::FeatureLinearPattern2](#).

.NET Syntax

Visual Basic (Declaration)

```
Function FeatureLinearPattern( _  
    ByVal Num1 As System.Integer, _  
    ByVal Spacing1 As System.Double, _
```

```

    ByVal Num2 As System.Integer, _
    ByVal Spacing2 As System.Double, _
    ByVal FlipDir1 As System.Boolean, _
    ByVal FlipDir2 As System.Boolean, _
    ByVal DName1 As System.String, _
    ByVal DName2 As System.String _
) As Feature

```

Visual Basic (Usage)



```

Dim instance As IFeatureManager
Dim Num1 As System.Integer
Dim Spacing1 As System.Double
Dim Num2 As System.Integer
Dim Spacing2 As System.Double
Dim FlipDir1 As System.Boolean
Dim FlipDir2 As System.Boolean
Dim DName1 As System.String
Dim DName2 As System.String
Dim value As Feature

```

```

value = instance.FeatureLinearPattern(Num1, Spacing1, Num2, Spacing2, FlipDir1, FlipDir2, DName1, DName2)

```

C#

```

Feature FeatureLinearPattern(
    System.int Num1,
    System.double Spacing1,
    System.int Num2,
    System.double Spacing2,
    System.bool FlipDir1,
    System.bool FlipDir2,
    System.string DName1,
    System.string DName2
)

```

C++/CLI

```

Feature^ FeatureLinearPattern(
& System.int Num1,
& System.double Spacing1,
& System.int Num2,
& System.double Spacing2,
& System.bool FlipDir1,
& System.bool FlipDir2,
& System.String^ DName1,
& System.String^ DName2
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Num1

Spacing1

Num2

Spacing2

FlipDir1

FlipDir2

DName1

DName2

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureLinearPattern](#).

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)

SOLIDWORKS API Help

[Send comments](#) on this topic.

FeatureLinearPattern2 Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureLinearPattern2 Method (IFeatureManager)

Obsolete. Superseded by [IFeatureManager::FeatureLinearPattern3](#).

.NET Syntax

Visual Basic (Declaration)

```
Function FeatureLinearPattern2( _  
    ByVal Num1 As System.Integer, _  
    ByVal Spacing1 As System.Double, _  
    ByVal Num2 As System.Integer, _  
    ByVal Spacing2 As System.Double, _  
    ByVal FlipDir1 As System.Boolean, _  
    ByVal FlipDir2 As System.Boolean, _  
    ByVal DName1 As System.String, _  
    ByVal DName2 As System.String, _  
    ByVal GeometryPattern As System.Boolean _  
) As Feature
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim Num1 As System.Integer  
Dim Spacing1 As System.Double  
Dim Num2 As System.Integer  
Dim Spacing2 As System.Double  
Dim FlipDir1 As System.Boolean  
Dim FlipDir2 As System.Boolean  
Dim DName1 As System.String  
Dim DName2 As System.String  
Dim GeometryPattern As System.Boolean  
Dim value As Feature
```

```
value = instance.FeatureLinearPattern2(Num1, Spacing1, Num2, Spacing2, FlipDir1, FlipDir2, DName1,  
DName2, GeometryPattern)
```

C#

```
Feature FeatureLinearPattern2(  
    System.int Num1,  
    System.double Spacing1,  
    System.int Num2,  
    System.double Spacing2,  
    System.bool FlipDir1,  
    System.bool FlipDir2,  
    System.string DName1,  
    System.string DName2,  
    System.bool GeometryPattern  
)
```

C++/CLI

```
Feature^ FeatureLinearPattern2(  
& System.int Num1,  
& System.double Spacing1,  
& System.int Num2,  
& System.double Spacing2,  
& System.bool FlipDir1,  
& System.bool FlipDir2,  
& System.String^ DName1,
```

```
& System.String^ DName2,  
& System.bool GeometryPattern  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Num1
Number of instances of the linear pattern in Direction 1, including the original
Spacing1
Spacing between each instance of the linear pattern in Direction 1 in meters
Num2
Number of instances of the linear pattern in Direction 2, including the original
Spacing2
Spacing between each instance of the linear pattern in Direction 2 in meters
FlipDir1
True if you want to reverse the direction the linear pattern in Direction 1, false if not
FlipDir2
True if you want to reverse the direction of the linear pattern in Direction 2, false if not
DName1
Name of the dimension defining Direction 1
DName2
Name of the dimension defining Direction 2
GeometryPattern
True to use geometry pattern, false to not

Return Value

[Feature](#)

 Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureLinearPattern2](#).

 Remarks

If...	To select a feature, use...	To select a component, use...
Using IModelDocExtension::SelectByID2 to select features and components, ordered selection of the features and components is required	• 1 and 2 to mark the directions • 4 to mark the features to pattern	• 1 to mark the components to pattern • 2 and 4 to mark the directions
Directly selecting a feature or component	IFeature::Select2	ISelectData::Mark and Component2::Select3

 See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[ILinearPatternFeatureData Interface](#)

 Availability

SOLIDWORKS 2005 FCS, Revision Number 13.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FeatureLinearPattern3 Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureLinearPattern3 Method (IFeatureManager)

Obsolete. Superseded by [IFeatureManager::FeatureLinearPattern4](#).

 .NET Syntax

Visual Basic (Declaration)

```
Function FeatureLinearPattern3( _  
    ByVal Num1 As System.Integer, _  
    ByVal Spacing1 As System.Double, _  
    ByVal Num2 As System.Integer, _  
    ByVal Spacing2 As System.Double, _  
    ByVal FlipDir1 As System.Boolean, _
```

```

        ByVal FlipDir2 As System.Boolean, _
        ByVal DName1 As System.String, _
        ByVal DName2 As System.String, _
        ByVal GeometryPattern As System.Boolean, _
        ByVal VaryInstance As System.Boolean _
    ) As Feature

```

Visual Basic (Usage)



```

Dim instance As IFeatureManager
Dim Num1 As System.Integer
Dim Spacing1 As System.Double
Dim Num2 As System.Integer
Dim Spacing2 As System.Double
Dim FlipDir1 As System.Boolean
Dim FlipDir2 As System.Boolean
Dim DName1 As System.String
Dim DName2 As System.String
Dim GeometryPattern As System.Boolean
Dim VaryInstance As System.Boolean
Dim value As Feature

```

```

value = instance.FeatureLinearPattern3(Num1, Spacing1, Num2, Spacing2, FlipDir1, FlipDir2, DName1,
DName2, GeometryPattern, VaryInstance)

```

C#

```

Feature FeatureLinearPattern3(
    System.int Num1,
    System.double Spacing1,
    System.int Num2,
    System.double Spacing2,
    System.bool FlipDir1,
    System.bool FlipDir2,
    System.string DName1,
    System.string DName2,
    System.bool GeometryPattern,
    System.bool VaryInstance
)

```

C++/CLI

```

Feature^ FeatureLinearPattern3(
    & System.int Num1,
    & System.double Spacing1,
    & System.int Num2,
    & System.double Spacing2,
    & System.bool FlipDir1,
    & System.bool FlipDir2,
    & System.String^ DName1,
    & System.String^ DName2,
    & System.bool GeometryPattern,
    & System.bool VaryInstance
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Num1

Number of instances of the linear pattern in Direction 1, including the original

Spacing1

Spacing between each instance of the linear pattern in Direction 1 in meters

Num2

Number of instances of the linear pattern in Direction 2, including the original

Spacing2

Spacing between each instance of the linear pattern in Direction 2 in meters

FlipDir1

True if you want to reverse the direction of the linear pattern in Direction 1, false if not

FlipDir2

True if you want to reverse the direction of the linear pattern in Direction 2, false if not

DName1

Name of the dimension defining Direction 1

DName2

Name of the dimension defining Direction 2

GeometryPattern

True to use geometry pattern, false to not

VaryInstance
True to vary the dimensions and spacing of individual pattern instances, false to not; valid only if GeometryPattern = false (see **Remarks**)

Return Value

[Feature](#)

 Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureLinearPattern3](#).

 Remarks

If...	To select a feature, use...	To select a component, use...
Using IModelDocExtension::SelectByID2 to select features and components, ordered selection of the features and components is required	• 1 and 2 to mark the directions • 4 to mark the features to pattern	• 1 to mark the components to pattern • 2 and 4 to mark the directions
Directly selecting a feature or component	IFeature::Select2	ISelectData::Mark and IComponent2::Select3

If VaryInstance = true, then to indicate how to vary the individual pattern instances, decide on the type of pattern and call its corresponding method before calling this method:

Type...	Method...
Increment	IFeatureManager::InsertVaryInstanceIncrement
Override	IFeatureManager::InsertVaryInstanceOverride

 See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[ILinearPatternFeatureData Interface](#)
[ILocalLinearPatternFeatureData Interface](#)

 Availability

SOLIDWORKS 2013 FCS, Revision Number 21.0

FeatureLinearPattern4 Method (IFeatureManager)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureLinearPattern4 Method (IFeatureManager)

Obsolete. Superseded by [IFeatureManager::FeatureLinearPattern5](#).

 .NET Syntax

Visual Basic (Declaration)

```
Function FeatureLinearPattern4( _
    ByVal Num1 As System.Integer, _
    ByVal Spacing1 As System.Double, _
    ByVal Num2 As System.Integer, _
    ByVal Spacing2 As System.Double, _
    ByVal FlipDir1 As System.Boolean, _
    ByVal FlipDir2 As System.Boolean, _
    ByVal DName1 As System.String, _
    ByVal DName2 As System.String, _
    ByVal GeometryPattern As System.Boolean, _
    ByVal VaryInstance As System.Boolean, _
    ByVal HasOffset1 As System.Boolean, _
    ByVal HasOffset2 As System.Boolean, _
    ByVal CtrlByNum1 As System.Boolean, _
    ByVal CtrlByNum2 As System.Boolean, _
    ByVal FromCentroid1 As System.Boolean, _
```

```

    ByVal FromCentroid2 As System.Boolean, _
    ByVal RevOffset1 As System.Boolean, _
    ByVal RevOffset2 As System.Boolean, _
    ByVal Offset1 As System.Double, _
    ByVal Offset2 As System.Double _
) As Feature

```

Visual Basic (Usage)



```

Dim instance As IFeatureManager
Dim Num1 As System.Integer
Dim Spacing1 As System.Double
Dim Num2 As System.Integer
Dim Spacing2 As System.Double
Dim FlipDir1 As System.Boolean
Dim FlipDir2 As System.Boolean
Dim DName1 As System.String
Dim DName2 As System.String
Dim GeometryPattern As System.Boolean
Dim VaryInstance As System.Boolean
Dim HasOffset1 As System.Boolean
Dim HasOffset2 As System.Boolean
Dim CtrlByNum1 As System.Boolean
Dim CtrlByNum2 As System.Boolean
Dim FromCentroid1 As System.Boolean
Dim FromCentroid2 As System.Boolean
Dim RevOffset1 As System.Boolean
Dim RevOffset2 As System.Boolean
Dim Offset1 As System.Double
Dim Offset2 As System.Double
Dim value As Feature

```

```

value = instance.FeatureLinearPattern4(Num1, Spacing1, Num2, Spacing2, FlipDir1, FlipDir2, DName1,
DName2, GeometryPattern, VaryInstance, HasOffset1, HasOffset2, CtrlByNum1, CtrlByNum2,
FromCentroid1, FromCentroid2, RevOffset1, RevOffset2, Offset1, Offset2)

```

C#

```

Feature FeatureLinearPattern4(
    System.int Num1,
    System.double Spacing1,
    System.int Num2,
    System.double Spacing2,
    System.bool FlipDir1,
    System.bool FlipDir2,
    System.string DName1,
    System.string DName2,
    System.bool GeometryPattern,
    System.bool VaryInstance,
    System.bool HasOffset1,
    System.bool HasOffset2,
    System.bool CtrlByNum1,
    System.bool CtrlByNum2,
    System.bool FromCentroid1,
    System.bool FromCentroid2,
    System.bool RevOffset1,
    System.bool RevOffset2,
    System.double Offset1,
    System.double Offset2
)

```

C++/CLI

```

Feature^ FeatureLinearPattern4(
& System.int Num1,
& System.double Spacing1,
& System.int Num2,
& System.double Spacing2,
& System.bool FlipDir1,
& System.bool FlipDir2,

```

```

& System.String^ DName1,
& System.String^ DName2,
& System.bool GeometryPattern,
& System.bool VaryInstance,
& System.bool HasOffset1,
& System.bool HasOffset2,
& System.bool CtrlByNum1,
& System.bool CtrlByNum2,
& System.bool FromCentroid1,
& System.bool FromCentroid2,
& System.bool RevOffset1,
& System.bool RevOffset2,
& System.double Offset1,
& System.double Offset2
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Num1

Number of instances of the linear pattern in Direction 1, including the original

Spacing1

Spacing in meters between each instance of the linear pattern in Direction 1

Num2

Number of instances of the linear pattern in Direction 2, including the original

Spacing2

Spacing in meters between each instance of the linear pattern in Direction 2

FlipDir1

True to reverse the direction of the linear pattern in Direction 1, false to not

FlipDir2

True to reverse the direction of the linear pattern in Direction 2, false to not

DName1

Name of the dimension defining Direction 1

DName2

Name of the dimension defining Direction 2

GeometryPattern

True to use geometry pattern, false to not

VaryInstance

True to vary the dimensions and spacing of individual pattern instances, false to not; valid only if GeometryPattern = false (see **Remarks**)

HasOffset1

True if using Offset1 to specify an offset of the pattern seed from a selected reference in Direction 1, false if not

HasOffset2

True if using Offset2 to specify an offset of the pattern seed from a selected reference in Direction 2, false if not

CtrlByNum1

True to control pattern spacing using Num1, false to control it using Spacing1; valid only if HasOffset1 is true

CtrlByNum2

True to control pattern spacing using Num2, false to control it using Spacing2; valid only if HasOffset2 is true

FromCentroid1

True if Offset1 is measured from the centroid of the seed instance, false if it is measured from a selected reference on the seed instance; valid only if HasOffset1 is true (see **Remarks**)

FromCentroid2

True if Offset2 is measured from the centroid of the seed instance, false if it is measured from a selected reference on the seed instance; valid only if HasOffset2 is true (see **Remarks**)

RevOffset1

True to reverse the direction of Offset1, false to not; valid only if HasOffset1 is true

RevOffset2

True to reverse the direction of Offset2, false to not; valid only if HasOffset2 is true

Offset1

Offset in meters from a selected reference in Direction 1 (see **Remarks**); valid only if HasOffset1 is true

Offset2

Offset in meters from a selected reference in Direction 2 (see **Remarks**); valid only if HasOffset2 is true

Return Value

Linear pattern [feature](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager.FeatureLinearPattern4](#).

Example

[Create Linear Pattern \(VBA\)](#)
[Create Linear Pattern \(VB.NET\)](#)
[Create Linear Pattern \(C#\)](#)

Remarks

If...	To select a feature, use...	To select a component, use...
Using IModelDocExtension::SelectByID2 to select features and components, ordered selection of the features and components is required	1 to mark Direction 1, and 2 to mark Direction 2 4 to mark the features to pattern 2097152 to mark references for Offset1 and Offset2	1 to mark the components to pattern 2 and 4 to mark the directions

If...	To select a feature, use...	To select a component, use...
Directly selecting a feature or component using IFeature::Select2	8388608 to mark a seed instance reference when FromCentroid1 or FromCentroid2 is false 1 to mark Direction 1, and 2 to mark Direction 2 4 to mark the features to pattern 2097152 to mark references for Offset1 and Offset2 8388608 to mark a seed instance reference when FromCentroid1 or FromCentroid2 is false	ISelectData::Mark and IComponent2::Select4 1 to mark the components to pattern 2 and 4 to mark the directions

If VaryInstance = true, then to indicate how to vary the individual pattern instances, decide on the type of pattern and call its corresponding method before calling this method:

Type	Method
Increment	IFeatureManager::InsertVaryInstanceIncrement
Override	IFeatureManager::InsertVaryInstanceOverride

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[ILinearPatternFeatureData Interface](#)
[ILocalLinearPatternFeatureData Interface](#)

Availability

SOLIDWORKS 2015 FCS, Revision Number 23.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FeatureLinearPattern5 Method (IFeatureManager)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureLinearPattern5 Method (IFeatureManager)

Obsolete. See [IFeatureManager::CreateFeature](#) and the Remarks in [ILinearPatternFeatureData](#) and [ILocalLinearPatternFeatureData](#).

.NET Syntax

Visual Basic (Declaration)

```

Function FeatureLinearPattern5( _
    ByVal Num1 As System.Integer, _
    ByVal Spacing1 As System.Double, _
    ByVal Num2 As System.Integer, _
    ByVal Spacing2 As System.Double, _
    ByVal FlipDir1 As System.Boolean, _
    ByVal FlipDir2 As System.Boolean, _
    ByVal DName1 As System.String, _
    ByVal DName2 As System.String, _
    ByVal GeometryPattern As System.Boolean, _
    ByVal VaryInstance As System.Boolean, _
    ByVal HasOffset1 As System.Boolean, _
    ByVal HasOffset2 As System.Boolean, _
    ByVal CtrlByNum1 As System.Boolean, _
    ByVal CtrlByNum2 As System.Boolean, _
    ByVal FromCentroid1 As System.Boolean, _
    ByVal FromCentroid2 As System.Boolean, _
    ByVal RevOffset1 As System.Boolean, _
    ByVal RevOffset2 As System.Boolean, _
    ByVal Offset1 As System.Double, _
    ByVal Offset2 As System.Double, _
    ByVal D2PatternSeedOnly As System.Boolean, _
    ByVal SyncSubAssemblies As System.Boolean _
) As Feature

```

```

Dim instance As IFeatureManager
Dim Num1 As System.Integer
Dim Spacing1 As System.Double
Dim Num2 As System.Integer
Dim Spacing2 As System.Double
Dim FlipDir1 As System.Boolean
Dim FlipDir2 As System.Boolean
Dim DName1 As System.String
Dim DName2 As System.String
Dim GeometryPattern As System.Boolean
Dim VaryInstance As System.Boolean
Dim HasOffset1 As System.Boolean
Dim HasOffset2 As System.Boolean
Dim CtrlByNum1 As System.Boolean
Dim CtrlByNum2 As System.Boolean
Dim FromCentroid1 As System.Boolean
Dim FromCentroid2 As System.Boolean
Dim RevOffset1 As System.Boolean
Dim RevOffset2 As System.Boolean
Dim Offset1 As System.Double
Dim Offset2 As System.Double
Dim D2PatternSeedOnly As System.Boolean
Dim SyncSubAssemblies As System.Boolean
Dim value As Feature

```

```

value = instance.FeatureLinearPattern5(Num1, Spacing1, Num2, Spacing2, FlipDir1, FlipDir2, DName1,
DName2, GeometryPattern, VaryInstance, HasOffset1, HasOffset2, CtrlByNum1, CtrlByNum2,
FromCentroid1, FromCentroid2, RevOffset1, RevOffset2, Offset1, Offset2, D2PatternSeedOnly,
SyncSubAssemblies)

```

C#

```

Feature FeatureLinearPattern5(
    System.int Num1,
    System.double Spacing1,
    System.int Num2,
    System.double Spacing2,
    System.bool FlipDir1,
    System.bool FlipDir2,
    System.string DName1,
    System.string DName2,
    System.bool GeometryPattern,
    System.bool VaryInstance,
    System.bool HasOffset1,
    System.bool HasOffset2,
    System.bool CtrlByNum1,
    System.bool CtrlByNum2,
    System.bool FromCentroid1,
    System.bool FromCentroid2,
    System.bool RevOffset1,
    System.bool RevOffset2,
    System.double Offset1,
    System.double Offset2,
    System.bool D2PatternSeedOnly,
    System.bool SyncSubAssemblies
)

```

C++/CLI

```

Feature^ FeatureLinearPattern5(
& System.int Num1,
& System.double Spacing1,
& System.int Num2,
& System.double Spacing2,
& System.bool FlipDir1,
& System.bool FlipDir2,
& System.String^ DName1,
& System.String^ DName2,

```

```

& System.bool GeometryPattern,
& System.bool VaryInstance,
& System.bool HasOffset1,
& System.bool HasOffset2,
& System.bool CtrlByNum1,
& System.bool CtrlByNum2,
& System.bool FromCentroid1,
& System.bool FromCentroid2,
& System.bool RevOffset1,
& System.bool RevOffset2,
& System.double Offset1,
& System.double Offset2,
& System.bool D2PatternSeedOnly,
& System.bool SyncSubAssemblies
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Num1

Number of instances of the linear pattern in Direction 1, including the original

Spacing1

Spacing in meters between each instance of the linear pattern in Direction 1

Num2

Number of instances of the linear pattern in Direction 2, including the original

Spacing2

Spacing in meters between each instance of the linear pattern in Direction 2

FlipDir1

True to reverse the direction of the linear pattern in Direction 1, false to not

FlipDir2

True to reverse the direction of the linear pattern in Direction 2, false to not

DName1

Name of the dimension defining Direction 1

DName2

Name of the dimension defining Direction 2

GeometryPattern

True to use geometry pattern, false to not

VaryInstance

True to vary the dimensions and spacing of individual pattern instances, false to not; valid only if GeometryPattern = false (see **Remarks**)

HasOffset1

True if using Offset1 to specify an offset of the pattern seed from a selected reference in Direction 1, false if not

HasOffset2

True if using Offset2 to specify an offset of the pattern seed from a selected reference in Direction 2, false if not

CtrlByNum1

True to control pattern spacing using Num1, false to control it using Spacing1; valid only if HasOffset1 is true

CtrlByNum2

True to control pattern spacing using Num2, false to control it using Spacing2; valid only if HasOffset2 is true

FromCentroid1

True if Offset1 is measured from the centroid of the seed instance, false if it is measured from a selected reference on the seed instance; valid only if HasOffset1 is true (see **Remarks**)

FromCentroid2

True if Offset2 is measured from the centroid of the seed instance, false if it is measured from a selected reference on the seed instance; valid only if HasOffset2 is true (see **Remarks**)

RevOffset1

True to reverse the direction of Offset1, false to not; valid only if HasOffset1 is true

RevOffset2

True to reverse the direction of Offset2, false to not; valid only if HasOffset2 is true

Offset1

Offset in meters from a selected reference in Direction 1; valid only if HasOffset1 is true (see **Remarks**)

Offset2

Offset in meters from a selected reference in Direction 2; valid only if HasOffset2 is true (see **Remarks**)

D2PatternSeedOnly

True to create a linear pattern in Direction 2 using the seed features only, without replicating the pattern instances of Direction 1; false to not

SyncSubAssemblies

True to move components in the patterned instances when components are moved in the seed flexible subassembly, false to not

Return Value

Linear pattern [feature](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureLinearPattern5](#).

Example

[Create Bidirectional Linear Pattern \(VBA\)](#)
[Create Bidirectional Linear Pattern \(VB.NET\)](#)
[Create Bidirectional Linear Pattern \(C#\)](#)

Remarks

If...	To select a feature, use...	To select a body, use...	To select a component, use...
Using IModelDocExtension::SelectByID2 to select features, solid bodies, and components requires ordered selection	1 to mark Direction 1 2 to mark Direction 2 4 to mark the features to pattern	1 to mark Direction 1 256 to mark the solid bodies to pattern	1 to mark the components to pattern

If...	To select a feature, use...	To select a body, use...	To select a component, use...
	2097152 to mark references for Offset1 and Offset2 8388608 to mark a seed instance reference when FromCentroid1 or FromCentroid2 is false		2 and 4 to mark the directions
Directly selecting a feature, solid body, or component	IEntity::Select4 with ISelectData::Mark= 1 to mark Direction 1 2 to mark Direction 2 IFeature::Select2 with Mark= 4 to mark the features to pattern 2097152 to mark references for Offset1 and Offset2 8388608 to mark a seed instance reference when FromCentroid1 or FromCentroid2 is false	IEntity::Select4 with ISelectData::Mark=1 to mark Direction 1 IBody2::Select2 with ISelectData::Mark=256 to mark the bodies to pattern	IComponent2::Select4 and ISelectData::Mark=1 to mark the components to pattern IEntity::Select4 and ISelectData::Mark=2 and 4 to mark the directions

Type	Method
Increment	IFeatureManager::InsertVaryInstanceIncrement
Override	IFeatureManager::InsertVaryInstanceOverride

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)

Availability

SOLIDWORKS 2017 FCS, Revision Number 25.0

SOLIDWORKS API Help [Send comments on this topic.](#)

FeatureLocalCurveDrivenPattern Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureLocalCurveDrivenPattern Method (IFeatureManager)

Obsolete. See [IFeatureManager::CreateFeature](#) and the Remarks in [ILocalCurvePatternFeatureData](#).

.NET Syntax

Visual Basic (Declaration)

```
Function FeatureLocalCurveDrivenPattern( _
    ByVal FlipDir1 As System.Boolean, _
    ByVal Num1 As System.Integer, _
    ByVal EqualSpacing1 As System.Boolean, _
    ByVal Spacing1 As System.Double, _
    ByVal ReferencePoint As System.Integer, _
    ByVal CurveMethod As System.Integer, _
    ByVal AlignMethod As System.Integer, _
    ByVal Direction2 As System.Boolean, _
    ByVal FlipDir2 As System.Boolean, _
    ByVal Num2 As System.Integer, _
    ByVal EqualSpacing2 As System.Boolean, _
    ByVal Spacing2 As System.Double, _
    ByVal PatternSeedOnly As System.Boolean _
) As Feature
```

Visual Basic (Usage)

```

Dim instance As IFeatureManager
Dim FlipDir1 As System.Boolean
Dim Num1 As System.Integer
Dim EqualSpacing1 As System.Boolean
Dim Spacing1 As System.Double
Dim ReferencePoint As System.Integer
Dim CurveMethod As System.Integer
Dim AlignMethod As System.Integer
Dim Direction2 As System.Boolean
Dim FlipDir2 As System.Boolean
Dim Num2 As System.Integer
Dim EqualSpacing2 As System.Boolean
Dim Spacing2 As System.Double
Dim PatternSeedOnly As System.Boolean
Dim value As Feature

```

```

value = instance.FeatureLocalCurveDrivenPattern(FlipDir1, Num1, EqualSpacing1, Spacing1,
ReferencePoint, CurveMethod, AlignMethod, Direction2, FlipDir2, Num2, EqualSpacing2, Spacing2,
PatternSeedOnly)

```

C#

```

Feature FeatureLocalCurveDrivenPattern(
    System.bool FlipDir1,
    System.int Num1,
    System.bool EqualSpacing1,
    System.double Spacing1,
    System.int ReferencePoint,
    System.int CurveMethod,
    System.int AlignMethod,
    System.bool Direction2,
    System.bool FlipDir2,
    System.int Num2,
    System.bool EqualSpacing2,
    System.double Spacing2,
    System.bool PatternSeedOnly
)

```

C++/CLI

```

Feature^ FeatureLocalCurveDrivenPattern(
    & System.bool FlipDir1,
    & System.int Num1,
    & System.bool EqualSpacing1,
    & System.double Spacing1,
    & System.int ReferencePoint,
    & System.int CurveMethod,
    & System.int AlignMethod,
    & System.bool Direction2,
    & System.bool FlipDir2,
    & System.int Num2,
    & System.bool EqualSpacing2,
    & System.double Spacing2,
    & System.bool PatternSeedOnly
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

FlipDir1

True flips the direction of the first direction, false does not

Num1

Number of instances in the first direction, including the original instance

EqualSpacing1

True uses equal spacing between instances in the first direction, false does not

Spacing1

Distance between pattern instances along the curve in the first direction when EqualSpacing1 is set to false

ReferencePoint

Type of selected reference point as defined in [swLocalCurvePatternReferencePoint_e](#) (see **Remarks**)

CurveMethod

Curve method as defined in [swLocalCurvePatternCurveMethod_e](#)

AlignMethod

Alignment method as defined in [swLocalCurvePatternAlignment_e](#)

Direction2
True creates the pattern in the second direction, false does not
FlipDir2
True flips the direction of the second direction, false does not
Num2
Number of instances in the second direction
EqualSpacing2
True uses equal spacing between instances in the second direction, false does not
Spacing2
Distance between pattern instances along the curve in the second direction when EqualSpacing2 is set to false
PatternSeedOnly
True replicates only the seed pattern, which was created under Direction 1, under Direction 2; false replicates all instances of the curve pattern, which was created under Direction 1, under Direction 2

Return Value

Local curve pattern [feature](#)

 Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureLocalCurveDrivenPattern](#).

 Remarks

To select...	Use a mark of...
Components to pattern	1
Curve, edge, sketch, or sketch entity for Direction 1	2
Curve, edge, sketch, or sketch entity for Direction 2	4
Reference point for ReferencePoint	32
NOTE: If ReferencePoint is set to swLocalSketchPatternReferencePoint_e.swLocalSketchPatternSelectedPoint, then the selected reference point must be a vertex.	
Face on which a 3D curve lies	64

 See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)

 Availability

SOLIDWORKS 2015 FCS, Revision Number 23.0

SOLIDWORKS API Help

[Send comments](#) on this topic.

FeatureLocalSketchDrivenPattern Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureLocalSketchDrivenPattern Method (IFeatureManager)

Obsolete. See [IFeatureManager::CreateFeature](#) and the Remarks in [JLocalSketchPatternFeatureData](#).

 .NET Syntax

Visual Basic (Declaration)

```
Function FeatureLocalSketchDrivenPattern( _  
    ByVal ReferencePoint As System.Integer _  
) As Feature
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager
Dim ReferencePoint As System.Integer
Dim value As Feature

value = instance.FeatureLocalSketchDrivenPattern(ReferencePoint)
```

C#

```
Feature FeatureLocalSketchDrivenPattern(
    System.int ReferencePoint
)
```

C++/CLI

```
Feature^ FeatureLocalSketchDrivenPattern(
    & System.int ReferencePoint
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

ReferencePoint
Type of selected reference point as defined in [swLocalSketchPatternReferencePoint_e](#) (see **Remarks**)

Return Value

Local sketch pattern [feature](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureLocalSketchDrivenPattern](#).

Remarks

To select...	Use a mark of...
Components to pattern	1
Sketch to define the pattern	16
Reference point for ReferencePoint	32
NOTE: If ReferencePoint is set to swLocalSketchPatternReferencePoint_e.swLocalSketchPatternSelectedPoint, then the selected reference point must be a vertex.	

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)

Availability

SOLIDWORKS 2015 FCS, Revision Number 23.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FeatureName Property (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureName Property (IFeatureManager)

Gets the feature name for the specified feature ID.

.NET Syntax

Visual Basic (Declaration)

```
ReadOnly Property FeatureName( _  
    ByVal NameID As System.Integer _  
) As System.String
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim NameID As System.Integer  
Dim value As System.String  
  
value = instance.FeatureName(NameID)
```

C#

```
System.string FeatureName(  
    System.int NameID  
) {get;}
```

C++/CLI

```
property System.String^ FeatureName {  
    System.String^ get(System.int NameID);  
}
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

NameID

Feature ID as defined in [swFeatureNameID_e](#)

Property Value

Feature name

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureName](#).

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[IFeature::GetTypeName Method](#)
[IFeatureManager::ShowFeatureName Property](#)
[IFeature::Name Property](#)

Availability

SOLIDWORKS 2005 FCS, Revision Number

SOLIDWORKS API Help

[Send comments](#) on this topic.

FeatureRevolve Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureRevolve Method (IFeatureManager)

Obsolete. Superseded by [IFeatureManager::FeatureRevolve2](#)

.NET Syntax

Visual Basic (Declaration)

```
Function FeatureRevolve( _  
    ByVal Angle As System.Double, _  
    ByVal ReverseDir As System.Boolean, _  
    ByVal Angle2 As System.Double, _  
    ByVal RevType As System.Integer, _  
    ByVal Options As System.Integer, _  
    ByVal Merge As System.Boolean, _  
    ByVal UseFeatScope As System.Boolean, _
```

```
ByVal UseAutoSel As System.Boolean _  
) As Feature
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim Angle As System.Double  
Dim ReverseDir As System.Boolean  
Dim Angle2 As System.Double  
Dim RevType As System.Integer  
Dim Options As System.Integer  
Dim Merge As System.Boolean  
Dim UseFeatScope As System.Boolean  
Dim UseAutoSel As System.Boolean  
Dim value As Feature
```

```
value = instance.FeatureRevolve(Angle, ReverseDir, Angle2, RevType, Options, Merge, UseFeatScope, UseAutoSel)
```

C#

```
Feature FeatureRevolve(  
    System.double Angle,  
    System.bool ReverseDir,  
    System.double Angle2,  
    System.int RevType,  
    System.int Options,  
    System.bool Merge,  
    System.bool UseFeatScope,  
    System.bool UseAutoSel  
)
```

C++/CLI

```
Feature^ FeatureRevolve(  
& System.double Angle,  
& System.bool ReverseDir,  
& System.double Angle2,  
& System.int RevType,  
& System.int Options,  
& System.bool Merge,  
& System.bool UseFeatScope,  
& System.bool UseAutoSel  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Angle

Angle of revolution in radians

ReverseDir

True reverses the angle direction, false does not

Angle2

Angle of revolution in radians

RevType

Type of revolution as defined in [swRevolveType_e](#)

Options

Additional control as defined in [swRevolveOptions_e](#)

Merge

True to merge the results in a multibody part, false to not

UseFeatScope

True if the feature only affects selected bodies, false if the feature affects all bodies

UseAutoSel

True to automatically select all bodies and have the feature affect those bodies, false to select the bodies or components the feature affects (see **Remarks**)

Return Value

Pointer to the [IFeature](#) object

Visual Basic for Applications (VBA) Syntax

See [FeatureManager.FeatureRevolve](#).

Remarks

To select an axis for a revolve feature, first call [IModelDocExtension::SelectByID2](#) with Mark set to 4. When UseAutoSel is false, the user must select the bodies or components that the feature will affect.

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[IRvolveFeatureData2 Interface](#)
[IFeatureManager::FeatureRevolveCut2 Method](#)
[IFeatureManager::FeatureRevolveThin Method](#)
[IFeatureManager::FeatureRevolveThinCut Method](#)

Availability

SOLIDWORKS 2003 FCS, Revision Number 11.0

SOLIDWORKS API Help

[Send comments](#) on this topic.

FeatureRevolve2 Method (IFeatureManager)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureRevolve2 Method (IFeatureManager)

Creates a base-, boss-, or cut-revolve feature.

Visual Basic Syntax

Visual Basic (Declaration)

```
Function FeatureRevolve2( _  
    ByVal SingleDir As System.Boolean, _  
    ByVal IsSolid As System.Boolean, _  
    ByVal IsThin As System.Boolean, _  
    ByVal IsCut As System.Boolean, _  
    ByVal ReverseDir As System.Boolean, _  
    ByVal BothDirectionUpToSameEntity As System.Boolean, _  
    ByVal Dir1Type As System.Integer, _  
    ByVal Dir2Type As System.Integer, _  
    ByVal Dir1Angle As System.Double, _  
    ByVal Dir2Angle As System.Double, _  
    ByVal OffsetReverse1 As System.Boolean, _  
    ByVal OffsetReverse2 As System.Boolean, _  
    ByVal OffsetDistance1 As System.Double, _  
    ByVal OffsetDistance2 As System.Double, _  
    ByVal ThinType As System.Integer, _  
    ByVal ThinThickness1 As System.Double, _  
    ByVal ThinThickness2 As System.Double, _  
    ByVal Merge As System.Boolean, _  
    ByVal UseFeatScope As System.Boolean, _  
    ByVal UseAutoSelect As System.Boolean _  
) As Feature
```

Visual Basic (Usage)

```
Dim instance As IFeatureManager  
Dim SingleDir As System.Boolean  
Dim IsSolid As System.Boolean  
Dim IsThin As System.Boolean  
Dim IsCut As System.Boolean  
Dim ReverseDir As System.Boolean  
Dim BothDirectionUpToSameEntity As System.Boolean  
Dim Dir1Type As System.Integer  
Dim Dir2Type As System.Integer  
Dim Dir1Angle As System.Double  
Dim Dir2Angle As System.Double  
Dim OffsetReverse1 As System.Boolean  
Dim OffsetReverse2 As System.Boolean  
Dim OffsetDistance1 As System.Double  
Dim OffsetDistance2 As System.Double  
Dim ThinType As System.Integer
```



```

Dim ThinThickness1 As System.Double
Dim ThinThickness2 As System.Double
Dim Merge As System.Boolean
Dim UseFeatScope As System.Boolean
Dim UseAutoSelect As System.Boolean
Dim value As Feature

value = instance.FeatureRevolve2(SingleDir, IsSolid, IsThin, IsCut, ReverseDir,
BothDirectionUpToSameEntity, Dir1Type, Dir2Type, Dir1Angle, Dir2Angle, OffsetReverse1,
OffsetReverse2, OffsetDistance1, OffsetDistance2, ThinType, ThinThickness1, ThinThickness2, Merge,
UseFeatScope, UseAutoSelect)

```

C#

```

Feature FeatureRevolve2(
    System.bool SingleDir,
    System.bool IsSolid,
    System.bool IsThin,
    System.bool IsCut,
    System.bool ReverseDir,
    System.bool BothDirectionUpToSameEntity,
    System.int Dir1Type,
    System.int Dir2Type,
    System.double Dir1Angle,
    System.double Dir2Angle,
    System.bool OffsetReverse1,
    System.bool OffsetReverse2,
    System.double OffsetDistance1,
    System.double OffsetDistance2,
    System.int ThinType,
    System.double ThinThickness1,
    System.double ThinThickness2,
    System.bool Merge,
    System.bool UseFeatScope,
    System.bool UseAutoSelect
)

```

C++/CLI

```

Feature^ FeatureRevolve2(
& System.bool SingleDir,
& System.bool IsSolid,
& System.bool IsThin,
& System.bool IsCut,
& System.bool ReverseDir,
& System.bool BothDirectionUpToSameEntity,
& System.int Dir1Type,
& System.int Dir2Type,
& System.double Dir1Angle,
& System.double Dir2Angle,
& System.bool OffsetReverse1,
& System.bool OffsetReverse2,
& System.double OffsetDistance1,
& System.double OffsetDistance2,
& System.int ThinType,
& System.double ThinThickness1,
& System.double ThinThickness2,
& System.bool Merge,
& System.bool UseFeatScope,
& System.bool UseAutoSelect
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

SingleDir

True if the revolve is in one direction, false if in two directions (see **Remarks**)

IsSolid

True if this is a solid revolve feature, false if not

IsThin

True if this is a thin revolve feature, false if not

IsCut

True if this is a cut revolve feature, false if not

ReverseDir

True reverses the angle of the revolution, false does not; only applies if Dir1Type is not swEndConditions_e.swEndCondMidPlane

BothDirectionUpToSameEntity

True if the revolve is up to the same entity in both directions, false if not; only applies if SingleDir is false and Dir1Type and Dir2Type are swEndConditions_e.swEndCondUpToVertex, swEndConditions_e.swEndCondUpToSurface, or swEndConditions_e.swEndCondOffsetFromSurface) (see **Remarks**)

Dir1Type

Revolve end condition as defined in [swEndConditions_e](#)

Dir2Type

Revolve end condition in direction 2; as defined in swEndConditions_e and only applies if Dir1Type is not swEndConditions_e.swEndCondMidPlane

Dir1Angle

Angle in radians of revolution in direction 1; only applies if Dir1Type is swEndConditions_e.swEndCondBlind

Dir2Angle

Angle in radians of revolution in direction 2; only applies if Dir2Type is swEndConditions_e.swEndCondBlind

OffsetReverse1

True to reverse the offset direction in direction 1, false to not; only applies if Dir1Type is swEndConditions_e.swEndCondOffsetFromSurface

OffsetReverse2

True to reverse the offset direction in direction 2, false to not; only applies if Dir2Type is swEndConditions_e.swEndCondOffsetFromSurface

OffsetDistance1

Offset distance in direction 1; only applies if Dir1Type is swEndConditions_e.swEndCondOffsetFromSurface

OffsetDistance2

Offset distance in direction 2; only applies if Dir2Type is swEndConditions_e.swEndCondOffsetFromSurface

ThinType

Type and direction as defined in [swThinWallType_e](#)

ThinThickness1

Wall thickness in direction 1 (if ThinType is swThinWallType_e.swThinWallMidPlane, (ThinThickness1)/2 is used for each direction)

ThinThickness2

Wall thickness in direction 2 (only applies if ThinType is swThinWallType_e.swThinWallTwoDirection)

Merge

True to merge the results into a multi-body part, false to not

UseFeatScope

True if the feature only affects selected bodies, false if the feature affects all bodies (see **Remarks**)

UseAutoSelect

True to automatically select all bodies and have the feature affect those bodies, false to select the bodies or components that the feature affects (see **Remarks**)

Return Value

[Feature](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureRevolve2](#).

Example

[Create Revolve Features \(VBA\)](#)

[Create Revolve Features \(VB.NET\)](#)

[Create Revolve Features \(C#\)](#)

[Create Thin Feature Revolve in Two Directions \(VBA\)](#)

[Create Thin Feature Revolve in Two Directions \(VB.NET\)](#)

[Create Thin Feature Revolve in Two Directions \(C#\)](#)

[Create 360-degree Revolve Feature \(VBA\)](#)

[Create 360-degree Revolve Feature \(VB.NET\)](#)

[Create 360-degree Revolve Feature \(C#\)](#)

Remarks

Before calling this method, call [IModelDocExtension::SelectById2](#) to select:

- the sketch to revolve, using Mark = 0.
- the axis of revolution, using Mark = 4 or 16.
- the up-to surface, up-to vertex, or offset-from surface, using Mark = 32.
- one or more affected bodies or components (only if UseFeatScope is set to true and UseAutoSel is set to false), using Mark = 1 for each.

If SingleDir is false and the revolve is up to or offset from the same entity in both directions, select the entity once and set BothDirectionUpToSameEntity to true. If you select the same entity for each direction, the second selection negates the first, and the revolve feature is not created.

NOTE: This method provides more functionality, i.e., additional end conditions, than the now obsolete [IFeatureManager::FeatureRevolveCut2](#), [IFeatureManager::FeatureRevolveThin](#), and [IFeatureManager::FeatureRevolveThinCut](#).

See Also

[IFeatureManager Interface](#)

[IFeatureManager Members](#)

[IRvolveFeatureData2 Interface](#)

Availability

SOLIDWORKS 2011 FCS, Revision Number 19.0



.NET Syntax

Visual Basic (Declaration)

```
Function FeatureRevolveCut( _  
    ByVal Angle As System.Double, _  
    ByVal ReverseDir As System.Boolean, _  
    ByVal Angle2 As System.Double, _  
    ByVal RevType As System.Integer, _  
    ByVal Options As System.Integer, _  
    ByVal UseFeatScope As System.Boolean, _  
    ByVal UseAutoSelect As System.Boolean _  
) As Feature
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim Angle As System.Double  
Dim ReverseDir As System.Boolean  
Dim Angle2 As System.Double  
Dim RevType As System.Integer  
Dim Options As System.Integer  
Dim UseFeatScope As System.Boolean  
Dim UseAutoSelect As System.Boolean  
Dim value As Feature
```

```
value = instance.FeatureRevolveCut(Angle, ReverseDir, Angle2, RevType, Options, UseFeatScope,  
UseAutoSelect)
```

C#

```
Feature FeatureRevolveCut(  
    System.double Angle,  
    System.bool ReverseDir,  
    System.double Angle2,  
    System.int RevType,  
    System.int Options,  
    System.bool UseFeatScope,  
    System.bool UseAutoSelect  
)
```

C++/CLI

```
Feature^ FeatureRevolveCut(  
& System.double Angle,  
& System.bool ReverseDir,  
& System.double Angle2,  
& System.int RevType,  
& System.int Options,  
& System.bool UseFeatScope,  
& System.bool UseAutoSelect  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Angle

Angle of revolution in radians

ReverseDir

True reverses the angle direction, false does not

Angle2

Angle of revolution in radians

RevType

Type of revolution as defined in [swRevolveType_e](#) (can be either mid-plane or two-direction)

Options

Additional control (see **Remarks**)

UseFeatScope

True if the feature only affects selected bodies, false if the feature affects all bodies

UseAutoSelect

True to automatically select all bodies and have the feature affect those bodies, false to select the bodies the feature affects (see **Remarks**)

Return Value

Pointer to the [IFeature](#) object

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureRevolveCut](#).

Example

[Create Revolve Features \(VBA\)](#)

Remarks

The Options argument allows additional control of the feature creation. Supported value is swAutoCloseSketch close the sketch if it is open.

When UseAutoSelect is false, the user must select the bodies that the feature will affect.

When using cut or cavity features that result in multiple bodies, you cannot select to keep all of the resulting bodies or one or more selected bodies.

For extruded feature cuts, see [IFeatureManager::FeatureCut](#).

See Also

[IFeatureManager Interface](#)

[IFeatureManager Members](#)

[IFeatureManager::FeatureRevolve Method](#)

[IFeatureManager::FeatureRevolveThin Method](#)

[IFeatureManager::FeatureRevolveThinCut Method](#)

[IRvolveFeatureData2 Interface](#)

Availability

SOLIDWORKS 2003 FCS, Revision Number 11.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FeatureRevolveCut2 Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureRevolveCut2 Method (IFeatureManager)

Obsolete. Superseded by [IFeatureManager::FeatureRevolve2](#).

.NET Syntax

Visual Basic (Declaration)

```
Function FeatureRevolveCut2( _  
    ByVal Angle As System.Double, _  
    ByVal ReverseDir As System.Boolean, _  
    ByVal Angle2 As System.Double, _  
    ByVal RevType As System.Integer, _  
    ByVal Options As System.Integer, _  
    ByVal UseFeatScope As System.Boolean, _  
    ByVal UseAutoSelect As System.Boolean, _  
    ByVal AssemblyFeatureScope As System.Boolean, _  
    ByVal AutoSelectComponents As System.Boolean, _  
    ByVal PropagateFeatureToParts As System.Boolean _  
) As Feature
```

Visual Basic (Usage)



```

Dim instance As IFeatureManager
Dim Angle As System.Double
Dim ReverseDir As System.Boolean
Dim Angle2 As System.Double
Dim RevType As System.Integer
Dim Options As System.Integer
Dim UseFeatScope As System.Boolean
Dim UseAutoSelect As System.Boolean
Dim AssemblyFeatureScope As System.Boolean
Dim AutoSelectComponents As System.Boolean
Dim PropagateFeatureToParts As System.Boolean
Dim value As Feature

```

```

value = instance.FeatureRevolveCut2(Angle, ReverseDir, Angle2, RevType, Options, UseFeatScope,
UseAutoSelect, AssemblyFeatureScope, AutoSelectComponents, PropagateFeatureToParts)

```

C#

```

Feature FeatureRevolveCut2(
    System.double Angle,
    System.bool ReverseDir,
    System.double Angle2,
    System.int RevType,
    System.int Options,
    System.bool UseFeatScope,
    System.bool UseAutoSelect,
    System.bool AssemblyFeatureScope,
    System.bool AutoSelectComponents,
    System.bool PropagateFeatureToParts
)

```

C++/CLI

```

Feature^ FeatureRevolveCut2(
    & System.double Angle,
    & System.bool ReverseDir,
    & System.double Angle2,
    & System.int RevType,
    & System.int Options,
    & System.bool UseFeatScope,
    & System.bool UseAutoSelect,
    & System.bool AssemblyFeatureScope,
    & System.bool AutoSelectComponents,
    & System.bool PropagateFeatureToParts
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Angle

Angle of revolution in radians

ReverseDir

True reverses the angle direction, false does not

Angle2

Angle of revolution in radians

RevType

Type of revolution as defined in [swRevolveType_e](#) (can be either mid-plane or two-direction)

Options

Additional control (see **Remarks**)

UseFeatScope

True if the feature only affects selected bodies, false if the feature affects all bodies

UseAutoSelect

True to automatically select all bodies and have the feature affect those bodies, false to select the bodies the feature affects (see **Remarks**)

AssemblyFeatureScope

True if the assembly feature only affects selected components in the assembly, false if the assembly feature affects all components in the assembly

AutoSelectComponents

True to auto-select all affected components, false to not (use the selected components)

PropagateFeatureToParts

True to propagate the assembly feature to the components, false to not

Return Value

[Feature](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureRevolveCut2](#).

Remarks

The Options argument allows additional control of the feature creation. Supported value is `swRevolveOptions_e.swAutoCloseSketch`, which closes the sketch if it is open. When `UseAutoSelect` is false, the user must select the bodies that the feature will affect.

When using cut or cavity features that result in multiple bodies, you cannot select to keep all of the resulting bodies or one or more selected bodies.

For extruded feature cuts, see [IFeatureManager::FeatureCut](#).

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[IFeatureManager::FeatureRevolve Method](#)
[IFeatureManager::FeatureRevolveThin Method](#)
[IFeatureManager::FeatureRevolveThinCut Method](#)
[IRvolveFeatureData2 Interface](#)

Availability

SOLIDWORKS 2009 FCS, Revision Number 17.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FeatureRevolveThin Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureRevolveThin Method (IFeatureManager)

Obsolete. Superseded by [IFeatureManager::FeatureRevolve2](#).

.NET Syntax

Visual Basic (Declaration)

```
Function FeatureRevolveThin( _  
    ByVal Angle As System.Double, _  
    ByVal ReverseDir As System.Boolean, _  
    ByVal Angle2 As System.Double, _  
    ByVal RevType As System.Integer, _  
    ByVal Thickness1 As System.Double, _  
    ByVal Thickness2 As System.Double, _  
    ByVal ReverseThinDir As System.Integer, _  
    ByVal Merge As System.Boolean, _  
    ByVal UseFeatScope As System.Boolean, _  
    ByVal UseAutoSelect As System.Boolean _  
) As Feature
```

Visual Basic (Usage)

```
Dim instance As IFeatureManager  
Dim Angle As System.Double  
Dim ReverseDir As System.Boolean  
Dim Angle2 As System.Double  
Dim RevType As System.Integer  
Dim Thickness1 As System.Double  
Dim Thickness2 As System.Double  
Dim ReverseThinDir As System.Integer  
Dim Merge As System.Boolean  
Dim UseFeatScope As System.Boolean  
Dim UseAutoSelect As System.Boolean  
Dim value As Feature
```



```
value = instance.FeatureRevolveThin(Angle, ReverseDir, Angle2, RevType, Thickness1, Thickness2, ReverseThinDir, Merge, UseFeatScope, UseAutoSelect)
```

C#

```
Feature FeatureRevolveThin(  
    System.double Angle,  
    System.bool ReverseDir,  
    System.double Angle2,  
    System.int RevType,  
    System.double Thickness1,  
    System.double Thickness2,  
    System.int ReverseThinDir,  
    System.bool Merge,  
    System.bool UseFeatScope,  
    System.bool UseAutoSelect  
)
```

C++/CLI

```
Feature^ FeatureRevolveThin(  
& System.double Angle,  
& System.bool ReverseDir,  
& System.double Angle2,  
& System.int RevType,  
& System.double Thickness1,  
& System.double Thickness2,  
& System.int ReverseThinDir,  
& System.bool Merge,  
& System.bool UseFeatScope,  
& System.bool UseAutoSelect  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Angle

Angle of revolution in radians

ReverseDir

True reverses the angle direction, false does not

Angle2

Angle of revolution in radians

RevType

Type of revolution as defined in [swRevolveType_e](#) (see **Remarks**)

Thickness1

Wall thickness (ReverseThinDir = 2 uses (Thickness1)/2 for each direction)

Thickness2

Wall thickness; use only ReverseThinDir=2

ReverseThinDir

Type and direction as defined in [swThinWallType_e](#)

Merge

True to merge the results in a multibody part, false to not

UseFeatScope

True if the feature only affects selected bodies, false if the feature affects all bodies

UseAutoSelect

True to automatically select all bodies and have the feature affect those bodies, false to select the bodies the feature affects (see **Remarks**)

Return Value

Pointer to the [IFeature](#) object

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureRevolveThin](#).

Remarks

The ReverseDir argument has no effect if a mid-plane revolution is specified.

When UseAutoSelect is false, the user must select the bodies that the feature will affect.

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)

[IFeatureManager::FeatureRevolve Method](#)
[IFeatureManager::FeatureRevolveCut2 Method](#)
[IFeatureManager::FeatureRevolveThinCut Method](#)
[IRvolveFeatureData2 Interface](#)

Availability

SOLIDWORKS 2003 FCS, Revision Number 11.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FeatureRevolveThinCut Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureRevolveThinCut Method (IFeatureManager)

Obsolete. Superseded by [IFeatureManager::FeatureRevolve2](#).

.NET Syntax

Visual Basic (Declaration)

```
Function FeatureRevolveThinCut( _  
    ByVal Angle As System.Double, _  
    ByVal ReverseDir As System.Boolean, _  
    ByVal Angle2 As System.Double, _  
    ByVal RevType As System.Integer, _  
    ByVal Thickness1 As System.Double, _  
    ByVal Thickness2 As System.Double, _  
    ByVal ReverseThinDir As System.Integer, _  
    ByVal UseFeatScope As System.Boolean, _  
    ByVal UseAutoSelect As System.Boolean _  
) As Feature
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim Angle As System.Double  
Dim ReverseDir As System.Boolean  
Dim Angle2 As System.Double  
Dim RevType As System.Integer  
Dim Thickness1 As System.Double  
Dim Thickness2 As System.Double  
Dim ReverseThinDir As System.Integer  
Dim UseFeatScope As System.Boolean  
Dim UseAutoSelect As System.Boolean  
Dim value As Feature
```

```
value = instance.FeatureRevolveThinCut(Angle, ReverseDir, Angle2, RevType, Thickness1, Thickness2,  
ReverseThinDir, UseFeatScope, UseAutoSelect)
```

C#

```
Feature FeatureRevolveThinCut(  
    System.double Angle,  
    System.bool ReverseDir,  
    System.double Angle2,  
    System.int RevType,  
    System.double Thickness1,  
    System.double Thickness2,  
    System.int ReverseThinDir,  
    System.bool UseFeatScope,  
    System.bool UseAutoSelect  
)
```

C++/CLI

```
Feature^ FeatureRevolveThinCut(  
& System.double Angle,  
& System.bool ReverseDir,  
& System.double Angle2,  
& System.int RevType,
```

```

& System.double Thickness1,
& System.double Thickness2,
& System.int ReverseThinDir,
& System.bool UseFeatScope,
& System.bool UseAutoSelect
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Angle

Angle of revolution in radians

ReverseDir

True reverses the angle direction, false does not

Angle2

Angle of revolution in radians

RevType

Type of revolution as defined in [swRevolveType_e](#) (see **Remarks**)

Thickness1

Wall thickness1 (ReverseThinDir = 2 uses (thickness1)/2 for each direction)

Thickness2

Wall thickness 2 (only used when reverseThinDir= 3)

ReverseThinDir

Type and direction as defined in [swThinWallType_e](#)

UseFeatScope

True if the feature only affects selected bodies, false if the feature affects all bodies

UseAutoSelect

True to automatically select all bodies and have the feature affect those bodies, false to select the bodies the feature affects (see **Remarks**)

Return Value

Pointer to the [IFeature](#) object

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FeatureRevolveThinCut](#).

Remarks

A constant thickness is given to the profile when revolving.

The ReverseDir argument has no effect if a mid-plane revolution is specified.

When UseAutoSelect is false, the user must select the bodies that the feature will affect.

When using cut or cavity features that result in multiple bodies, you cannot select to keep all of the resulting bodies or one or more selected bodies.

See Also

[IFeatureManager Interface](#)

[IFeatureManager Members](#)

[IFeatureManager::FeatureRevolve Method](#)

[IFeatureManager::FeatureRevolveCut2 Method](#)

[IFeatureManager::FeatureRevolveThinCut Method](#)

[IRvolveFeatureData2 Interface](#)

Availability

SOLIDWORKS 2003 FCS, Revision Number 11.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FeatureSketchDrivenPattern Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FeatureSketchDrivenPattern Method (IFeatureManager)

Obsolete. See [IFeatureManager::CreateFeature](#) and the Remarks in [ISketchPatternFeatureData](#).

.NET Syntax

Visual Basic (Declaration)

```

Function FeatureSketchDrivenPattern( _
    ByVal UseCentroid As System.Boolean, _

```

```
ByVal BGeomPatt As System.Boolean _  
) As Feature
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim UseCentroid As System.Boolean  
Dim BGeomPatt As System.Boolean  
Dim value As Feature  
  
value = instance.FeatureSketchDrivenPattern(UseCentroid, BGeomPatt)
```

C#

```
Feature FeatureSketchDrivenPattern(  
    System.bool UseCentroid,  
    System.bool BGeomPatt  
)
```

C++/CLI

```
Feature^ FeatureSketchDrivenPattern(  
&    System.bool UseCentroid,  
&    System.bool BGeomPatt  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

UseCentroid

True to use a centroid as the reference point, false to not

BGeomPatt

True to pattern using only the feature geometry, false to pattern each instance

Return Value

Pointer to the [IFeature](#) object

Visual Basic for Applications (VBA) Syntax

See [FeatureManager.FeatureSketchDrivenPattern](#).

Remarks

Before calling this method, select the required entities using the proper selection marks:

- features = 4
- points = 32
- sketches = 64
- faces = 128
- bodies = 256

See [IModelDocExtension::SelectById2](#) for details about selecting input entities. See the SOLIDWORKS Help for more information about sketch-driven patterns.

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)

Availability

SOLIDWORKS 2003 SP4, Revision Number 11.4



Gets statistics about the features in a part document.

.NET Syntax

Visual Basic (Declaration)

ReadOnly Property FeatureStatistics As FeatureStatistics

Visual Basic (Usage)



```
Dim instance As IFeatureManager
Dim value As FeatureStatistics

value = instance.FeatureStatistics
```

C#

```
FeatureStatistics FeatureStatistics {get;}
```

C++/CLI

```
property FeatureStatistics^ FeatureStatistics {
    FeatureStatistics^ get();
}
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Property Value

[Feature statistics](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager.FeatureStatistics](#).

Example

[Get Part's Feature Statistics \(VB.NET\)](#)
[Get Part's Feature Statistics \(VBA\)](#)
[Get Part's Feature Statistics \(C#\)](#)

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)

Availability

SOLIDWORKS 2010 FCS, Revision Number 18.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FilletXpertChange Method (IFeatureManager)

[See Also](#)



SolidWorks.Interop.sldworks Namespace > IFeatureManager Interface : FilletXpertChange Method (IFeatureManager)

Changes the parameters on the selected filleted faces, regardless of whether the filleted faces were created manually or with FilletXpert, provided that FilletXpert can process them.

.NET Syntax

Visual Basic (Declaration)

```
Function FilletXpertChange( _
    ByVal Options As System.Integer, _
    ByVal R1 As System.Double, _
    ByVal Ftyp As System.Integer, _
    ByVal OverflowType As System.Integer _
) As Feature
```

```

Dim instance As IFeatureManager
Dim Options As System.Integer
Dim R1 As System.Double
Dim Ftyp As System.Integer
Dim OverflowType As System.Integer
Dim value As Feature

value = instance.FilletXpertChange(Options, R1, Ftyp, OverflowType)

```

C#

```

Feature FilletXpertChange(
    System.int Options,
    System.double R1,
    System.int Ftyp,
    System.int OverflowType
)

```

C++/CLI

```

Feature^ FilletXpertChange(
& System.int Options,
& System.double R1,
& System.int Ftyp,
& System.int OverflowType
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Options

Feature fillet option as defined in [swFeatureFilletOptions_e](#)

R1

Radius for uniform radius edge fillet

Ftyp

Type of fillet as defined in [swFeatureFilletType_e](#)

OverflowType

Control of fillet overflowing onto adjacent surfaces as defined in [swFilletOverFlowType_e](#)

Return Value

[IFeature](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FilletXpertChange](#).

See Also

[IFeatureManager Interface](#)

[IFeatureManager Members](#)

[IFeatureManager::FeatureFillet Method](#)

[IFeatureManager::FilletXpertRemove Method](#)

Availability

SOLIDWORKS 2007 FCS, Revision Number 15.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FilletXpertMakeCorner Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FilletXpertMakeCorner Method (IFeatureManager)

Creates or changes a fillet corner feature.

.NET Syntax

Visual Basic (Declaration)

```
Function FilletXpertMakeCorner( _  
    ByVal CornerType As System.Integer _  
) As Feature
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim CornerType As System.Integer  
Dim value As Feature  
  
value = instance.FilletXpertMakeCorner(CornerType)
```

C#

```
Feature FilletXpertMakeCorner(  
    System.int CornerType  
)
```

C++/CLI

```
Feature^ FilletXpertMakeCorner(  
& System.int CornerType  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

CornerType
See **Remarks**

Return Value

[Feature](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FilletXpertMakeCorner](#).

Remarks

For the fillet corner, everything is dependent on the convexity of the three filleted edges adjacent.

If all three edges have the same convexity, then no corner substitution is possible. Otherwise, there will be two edges of one convexity, and one of the other. It does not matter if the two same edges are both convex or both concave, as long as the edges are the same.

The valid values for the *CornerType* argument are:

- 1 = if the two edges with the same convexity have the same radius, make a triangular corner patch.
- 2 = if the two edges with the same convexity have the same radius, make a quadrilateral corner patch.
- 3 = if the two edges with the same convexity have different radii, make a triangular corner patch.
- 4 = if the two edges with the same convexity have different radii, make two triangular corner patches (which together look like a split quadrilateral patch).
- 5 = if the two edges with the same convexity have different radii, make a quadrilateral corner patch.

NOTE: Types 1 and 3 relate to one another, as do types 2 and 5. This means that if the corner type is not consistent with the convexity/radius requirements, then the appropriate type will be mapped and substituted. Thus, specifying type 1 on a corner with two edges of the same convexity, but different radii, results in a type 3, automatically.

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[IFeatureManager::FeatureFillet Method](#)
[IFeatureManager::IFeatureFillet Method](#)
[IFeatureManager::FilletXpertChange Method](#)
[IFeatureManager::FilletXpertRemove Method](#)

Availability

SOLIDWORKS 2008 FCS, Revision Number 16.0

FilletXpertRemove Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FilletXpertRemove Method (IFeatureManager)

Deletes the fillets on the selected faces.

.NET Syntax

Visual Basic (Declaration)

```
Function FilletXpertRemove() As Feature
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim value As Feature  
  
value = instance.FilletXpertRemove()
```

C#

```
Feature FilletXpertRemove()
```

C++/CLI

```
Feature^ FilletXpertRemove();
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Return Value

Pointer to [IFeature](#) object

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FilletXpertRemove](#).

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[IFeatureManager::FeatureFillet Method](#)
[IFeatureManager::FilletXpertChange Method](#)

Availability

SOLIDWORKS 2007 FCS, Revision Number 15.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FinishCornerRelief Method (IFeatureManager)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FinishCornerRelief Method (IFeatureManager)

Creates a sheet metal corner relief feature.

.NET Syntax

Visual Basic (Declaration)

```
Function FinishCornerRelief() As Feature
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim value As Feature  
  
value = instance.FinishCornerRelief()
```

C#

```
Feature FinishCornerRelief()
```

C++/CLI

```
Feature^ FinishCornerRelief();
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Return Value

[IFeature](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FinishCornerRelief](#).

Example

[Create Corner Relief Feature \(C#\)](#)
[Create Corner Relief Feature \(VBA\)](#)
[Create Corner Relief Feature \(VB.NET\)](#)

Remarks

Before calling this method, call:

1. [IModelDocExtension::SelectByID2](#) with Mark = 0 and Append = true to select the sheet metal body in which to create a corner relief feature.
2. [IModelDocExtension::SelectByID2](#) with Mark = 4 and Append = true to select two faces that form a bend corner.
3. Call [IFeatureManager::AddCornerReliefCorner](#) to add the corner to the corner relief feature.
4. Call [IFeatureManager::AddCornerReliefType](#) to specify the corner relief for the corner.
5. Repeat steps 2 - 4 to add another corner to the corner relief feature.

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[IFlatPatternFeatureData Interface](#)

Availability

SOLIDWORKS 2014 FCS, Revision Number 22.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

FinishSMNormalCut Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : FinishSMNormalCut Method (IFeatureManager)

Obsolete. See [IFeatureManager::CreateFeature](#) and the Remarks in [ISMNormalCutGroupData](#), and [ISMNormalCutFeatureData2](#).

.NET Syntax

Visual Basic (Declaration)

```
Function FinishSMNormalCut() As System.Object
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim value As System.Object  
  
value = instance.FinishSMNormalCut()
```

C#

```
System.object FinishSMNormalCut()
```

C++/CLI

```
System.Object^ FinishSMNormalCut();
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Return Value

[IFeature](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::FinishSMNormalCut](#).

Remarks

To create a sheet metal Normal Cut feature, see the [IFeatureManager::AddSMNormalCutType](#) Remarks and Example sections.

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[IFeatureManager::AddSMNormalCut Method \(\)](#)
[ISMNormalCutFeatureData Interface](#)

Availability

SOLIDWORKS 2018 FCS, Revision Number 26.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetCornerManagementFolders Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : GetCornerManagementFolders Method (IFeatureManager)

Gets the structure system corner management folders.

.NET Syntax

Visual Basic (Declaration)

```
Function GetCornerManagementFolders() As System.Object
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim value As System.Object
```

```
value = instance.GetCornerManagementFolders()
```

C#

```
System.object GetCornerManagementFolders()
```

C++/CLI

```
System.Object^ GetCornerManagementFolders();
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Return Value

Array of [IFeatures](#)

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::GetCornerManagementFolders](#).

Example

See the [ICornerManagementFolder](#) examples.

Remarks

The features returned are [ICornerManagementFolders](#). Use [IFeature::GetSpecificFeature2](#) after calling this method to get the ICornerManagementFolders.

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)

Availability

SOLIDWORKS 2023 FCS, Revision Number 31

SOLIDWORKS API Help

[Send comments](#) on this topic.

GetCreateFeatureErrors Method (IFeatureManager)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : GetCreateFeatureErrors Method (IFeatureManager)

Gets the errors that occurred during the last call to [IFeatureManager::CreateFeature](#).

.NET Syntax

Visual Basic (Declaration)

```
Function GetCreateFeatureErrors( _  
    ByRef Msgs As System.Object, _  
    ByRef FeatureTypeName As System.String _  
) As System.Integer
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager  
Dim Msgs As System.Object  
Dim FeatureTypeName As System.String  
Dim value As System.Integer  
  
value = instance.GetCreateFeatureErrors(Msgs, FeatureTypeName)
```

C#

```
System.int GetCreateFeatureErrors(  
    out System.object Msgs,  
    out System.string FeatureTypeName  
)
```

C++/CLI

```
System.int GetCreateFeatureErrors(  
& [Out] System.Object^ Msgs,  
& [Out] System.String^ FeatureTypeName  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Msgs

Array of errors as defined by [swCreateFeatureError_e](#)

FeatureTypeName

Feature type name

Return Value

0 <= Number of messages issued at the time of last call to IFeatureManager::CreateFeature <= 20

Visual Basic for Applications (VBA) Syntax

Example

'VBA

```
' =====
' Preconditions:
' 1. Open Public_documents\SOLIDWORKS\SOLIDWORKS 2024\samples\tutorial\api\AdvancedMates\advancedmatedemo1.sldasm.
' 2. Create two distance mates to center the steel disc inside the magnet.
'
' Postconditions:
' 1. Inspect the Immediate window.
' 2. Mate controller feature is not created.
' 3. Observe the returned error code: 6 indicates that the number of mate selections (2)
'    do not match the number of values (3) in the specified position data array.
' =====
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim Assem As SldWorks.AssemblyDoc
Dim boolstatus As Boolean
Dim i As Long, j As Long, longstatus As Long, longwarnings As Long
Option Explicit

Sub main()

    Set swApp = Application.SldWorks

    Set Part = swApp.ActiveDoc
    Set Assem = Part

    Dim myModelView As Object
    Set myModelView = Part.ActiveView
    myModelView.FrameState = swWindowState_e.swWindowMaximized
    Part.ClearSelection2 True

    Dim var As Variant
    var = Assem.CollectAllSupportiveMates

    Debug.Print "Mates:"
    For i = 0 To UBound(var)
        var(i).Select2 True, 0
        Debug.Print " " & var(i).Name
    Next

    Dim featMgr As SldWorks.FeatureManager
    Set featMgr = Part.FeatureManager

    Dim mateControllerObj As MateControllerFeatureData
    Set mateControllerObj = featMgr.CreateDefinition(swFmMateController)

    mateControllerObj.AddNewPosition ("Position1")
    mateControllerObj.AddNewPosition ("Position2")
    mateControllerObj.AddNewPosition ("Position3")

    Dim position2ValArr(2) As Double
    position2ValArr(0) = 1.0646
    position2ValArr(1) = 0.0635
    position2ValArr(2) = 1.02

    Call mateControllerObj.SetValues("Position2", position2ValArr)

    Dim posVar As Variant
    Dim valArr As Variant
    posVar = mateControllerObj.GetPositions()
    valArr = mateControllerObj.GetValues("Position3")

    Dim position3ValArr(2) As Double
    position3ValArr(0) = 1#
    position3ValArr(1) = 0.0635
    position3ValArr(2) = 1.05

    Call mateControllerObj.SetValues("Position3", position3ValArr)

    mateControllerObj.AddConfigurationFromPosition "Position 2", False

    mateControllerObj.DeletePosition ("Position 1")

    Dim posArr(2) As String
    posArr(0) = "Position3"
    posArr(1) = "Position2"
```

```
posArr(2) = "Position1"

mateControllerObj.ReOrderPositions ((posArr))

Dim mateContFeat As SldWorks.Feature
Set mateContFeat = featMgr.CreateFeature(mateControllerObj)
Dim err As Variant
Dim FeatureType As String
Dim cnt As Long
cnt = featMgr.GetCreateFeatureErrors(err, FeatureType)
Debug.Print "Number of errors returned: " & cnt
Debug.Print "Feature type: " & FeatureType

For j = 0 To UBound(err)
    Debug.Print "Error code as defined by swCreateFeatureError_e: " & err(j)
Next

End Sub
```

Remarks

This method is valid only after attempting to create [Mate Controller](#) features using IFeatureManager::CreateFeature.

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)

Availability

SOLIDWORKS 2024 FCS, Revision Number 32

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetFeatureCount Method (IFeatureManager)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [IFeatureManager Interface](#) : GetFeatureCount Method (IFeatureManager)

Gets the number of features in this document.

.NET Syntax

Visual Basic (Declaration)

```
Function GetFeatureCount( _
    ByVal ToplevelOnly As System.Boolean _
) As System.Integer
```

Visual Basic (Usage)



```
Dim instance As IFeatureManager
Dim ToplevelOnly As System.Boolean
Dim value As System.Integer
```

```
value = instance.GetFeatureCount(ToplevelOnly)
```

C#

```
System.int GetFeatureCount(
    System.bool ToplevelOnly
)
```

C++/CLI

```
System.int GetFeatureCount(
    & System.bool ToplevelOnly
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

TopLevelOnly

True to get only the number of features at the top level of the FeatureManager design tree, false to get the number of top-level features and all child features in the FeatureManager design tree

Return Value

Number of features

Visual Basic for Applications (VBA) Syntax

See [FeatureManager::GetFeatureCount](#).

Example

[Create Loft Body \(VB.NET\)](#)
[Create Loft Body \(VBA\)](#)
[Create Loft Body \(C#\)](#)

Remarks

Call this method before calling [IFeatureManager::GetFeatures](#) to determine the size of the array to pass into that method.

If TopLevelOnly is false, then this method counts all of the feature and child features in this document. It does not count features in components

See Also

[IFeatureManager Interface](#)
[IFeatureManager Members](#)
[IFeature::GetFeatures Method](#)
[IFeature::GetChildren Method](#)
[IFeature::GetFirstSubFeature Method](#)
[IFeature::GetNextFeature Method](#)
[IFeature::GetNextSubFeature Method](#)
[IFeature::GetOwnerFeature Method](#)
[IFeature::GetParents Method](#)
[IFeature::IGetChildCount Method](#)
[IFeature::IGetChildren Method](#)
[IFeature::IGetFirstSubFeature Method](#)
[IFeature::IGetNextFeature Method](#)
[IFeature::IGetNextSubFeature Method](#)
[IFeature::IGetParentCount Method](#)
[IFeature::IGetParents Method](#)
[IModelDoc2::FirstFeature Method](#)
[IModelDoc2::GetFeatureCount Method](#)
[IPartDoc::FirstFeature Method](#)
[IComponent2::FirstFeature Method](#)
[IModelDoc2::IFirstFeature Method](#)
[IPartDoc::IFirstFeature Method](#)

Availability

SOLIDWORKS 2006 FCS, Revision Number 14.0